FDE MODULE 2 | CLASS 2 | INSTRUCTOR SCRIPT (BEGINNER EDITION)

Your Workbench: Toolchain & Git Basics

Turning the Class 1 script into a cloneable, reproducible repository that another engineer can install, run and inspect safely

Class at a glance

Duration	120 minutes (quizzes and image moments are included in the section minutes).
Class purpose	Move the V0 Meridian Case Intake Workbench out of a single practice file and into a real project folder with its own Python environment, pinned dependencies, settings, safe handling of secrets, Git history and a README. By the end, another engineer can clone it, install it, run it and read the result.
Learner outcome	Learners can create and activate a virtual environment, install pinned packages, read a pyproject.toml, select the interpreter in VS Code, run ruff, keep settings and secrets out of the code, use Git (init, clone, status, add, commit, pull, push), and write a README that a partner can follow without help.
Starting point	The finished Class 1 program (Class 1 `attention_report.py`, with main and the if __name__ guard), the three Class 1 data files, and a VS Code window. Learners need Python 3.11 or newer and Git installed, and a free GitHub account. Run the prep checklist below at least one day before class.
Business anchor	The ops lead at Meridian asks: "Which cases need attention today, and what is the evidence?" The answer must now be reproducible by someone who was not in the room.
Deliverable	A GitHub repository called meridian-workbench that a partner has cloned and run from the README alone. It also contains an AI boundary that works with recorded responses and needs no API key.
Not in this class	No Streamlit build, no semantic evaluation framework, no provider-specific API lesson, no pytest or mypy (Class 3), no branches or pull requests, no Docker or CI (Class 7). The first live model call is in Class 3.
Tools	VS Code, a terminal (macOS/Linux shell or Windows PowerShell), Python 3.11+, Git, a GitHub account, a partner in the room or in a breakout.

Class timeline

Quiz time and image moments are inside the section minutes. The clock column is the target time at the end of each section.

Clock	Min	Section	Learners leave with
0-5	5	1. Welcome and the handoff	A felt reason for the class: the Class 1 file fails on a clean folder
5-12	7	2. Give the code a home	A project folder with src, tests, data, config, fixtures and outputs
12-22	10	3. Your own Python: the virtual environment	An activated .venv, proved with a command
22-37	15	4. pip, pinned dependencies and pyproject.toml	Pinned installs, requirements.txt, an editable install. Quiz 1
37-50	13	5. Split the script into modules	loaders, rules, results, pipeline, run with python -m. Quiz 2

Clock	Min	Section	Learners leave with
50-57	7	6. VS Code interpreter and ruff	The right interpreter selected, ruff check and format
57-70	13	7. Settings, environment variables and secrets	settings.toml, config.py, .env, .gitignore. Quiz 3
70-75	5	8. A place for proof: tests	tests/check_rules.py, broken on purpose
75-93	18	9. Git basics	A first commit, pushed to GitHub, then a clone and a pull. Quiz 4
93-105	12	10. README and the stranger test	A README a partner follows to a working run. Quiz 5
105-113	8	11. Prepare the AI boundary	ai_client.py, recorded fixture, boundary test. Quiz 6
113-120	7	12. Say it, then close	Three-sentence explanation, exit ticket, Class 3 preview

Instructor prep checklist

- Python and VS Code (with the Python extension) were set up in the Class 1 pre-read. One day ahead, ask learners to install Git and create a free GitHub account, then reply with screenshots of python --version (macOS/Linux: python3 --version, Windows: py --version) and git --version (Git 2.28 or newer, because we use git init -b main). Also ask them to confirm VS Code still opens and that Terminal > New Terminal works.
- Make sure every learner has the finished Class 1 program (with main and the guard) and the three data files. The Class 2 starter folder holds only data/ and fixtures/, so add the Class 1 final attention_report.py to it as a fallback for anyone who lost theirs. Section 1 and Section 5 both need it.
- Create a throwaway public GitHub repository called meridian-workbench in your account for the demo. Leave it empty (no README, no .gitignore, no licence).
- Decide how learners will sign in to GitHub from the terminal (Git Credential Manager, GitHub CLI, or a personal access token) and try it once on each operating system. Git sign-in is the most common time sink in this class.
- Prepare the six quizzes in your polling tool, or plan to post them in chat.

Starter data files

The same three files as Class 1. Copy cases.csv and notes.json into data/, and recorded_responses.json into fixtures/.

Download the Starter Files from the provided link.

Starter File Directory Link: Starter Files for Class 2

Starter Files Folder Structure: Starter Files Structure (Class 2)

The finished repository

Every file below was run and checked end to end in a clean environment. Learners who fall behind can read these instead of typing. If their output matches the expected output in Sections 5, 7, 8 and 11, they are fine. The settings.toml and config.py shown here are the final versions, including the [ai] additions from Section 11.

Download the final directory from the provided link. Use it if needed and for final check.

Final Files Directory Link: Final Directory after Class 2

Final Directory Folder Structure: Final Directory Structure (Class 2)

In the README, the sections are written in Markdown. The lines made of three backticks open and close code blocks.

Section 1: Welcome and the handoff

Duration	5 minutes (minute 0 to 5)
Teaching objective	Make learners feel the problem before any tool appears: the Class 1 program only works on the machine where it was written.

Say

> Welcome back. In Class 1 you wrote a program that answers one question for the Meridian ops lead: which cases need attention today, and what is the evidence. It works. Today the ops lead asks a harder question: can someone else run it? Not you, not on your laptop. A colleague, next week, on a different machine.
>
> Let me show you what happens today if I just send them the file.

Do: the stranger demo (2 minutes)

- Create an empty folder on your desktop called stranger_test and open a terminal in it.
- Copy only attention_report.py into it. Do not copy the data folder.
- Run the file. (On Windows, type `py` instead of `python3`.)

```
python3 attention_report.py
```

What Python prints

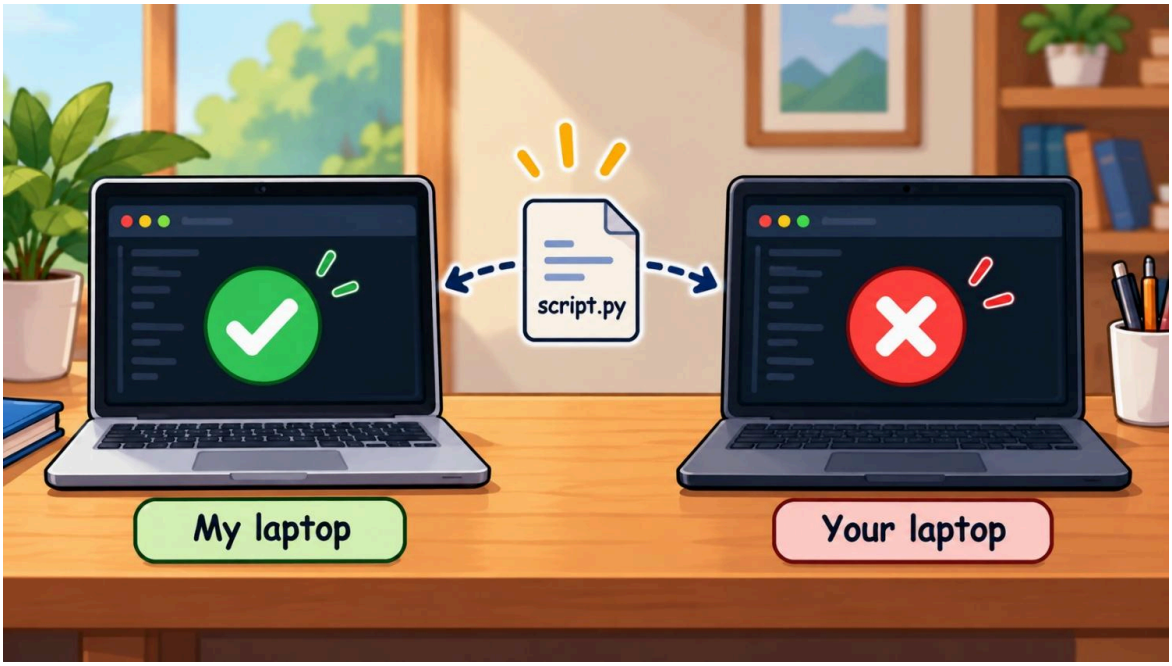
```
Traceback (most recent call last):
  ...
FileNotFoundError: [Errno 2] No such file or directory: 'data/cases.csv'
```

That block is called a **traceback**. It is Python's report of what went wrong and where. Do not be afraid of it. **Read the last line first.** It names the problem. Here it says `FileNotFoundError`: Python looked for `data/cases.csv` and it was not there.

Why not there? For two separate reasons. First, I never sent the data: the program needs `data/cases.csv`, and my colleague does not have it. Second, even with the data, `data/cases.csv` is a **relative path**, meaning "a folder called data inside the folder I am standing in", so the program only works when it is started from exactly the right folder. Hold on to both ideas. The repository fixes the first (Section 9), and a path that starts from the project folder fixes the second (Section 7).

> **Pause and predict**
> Before running it, ask: "I sent a colleague the file. What do you think happens?" Take two answers, then run it.

> The program is correct. The rules are correct. It still fails, because it depends on things that are on my laptop and nowhere else. Here it is the data folder. In bigger programs it is also the right version of Python and packages I installed months ago and forgot about. Programmers call this "works on my machine", and it is a joke for a reason.



A program is reproducible when someone else can set it up and run it, and get the same result as you, just by following written steps. That is our goal for today.

Today's title has the word toolchain. A toolchain is the set of tools you use one after another to get from code on your laptop to something others can run: an editor, a terminal, a private Python, a package installer, a checker, Git. Each does one job. We will meet them in order.

One more word you will hear all day: a dependency is anything your program needs that it did not create itself. That includes other packages, a particular Python version, and data files. "Works on my machine" happens when the dependencies are in your head instead of written down.

We will turn the Class 1 script into a proper project folder. This folder will have three things. First, a clear place for every file, so nothing is lost. Second, a list of exactly what the program needs, so nothing is missing. Third, a saved record of every change we make, so we can see what changed and go back if something breaks. That saved record is what we mean by history. We build it with a tool called Git, in Section 9.

When all three are in place, a colleague can download the folder, install it, run it, and check the result, without calling you for help.

The five-step map

Step	What we do	What it fixes
1	Give the code a home	Files scattered everywhere become a clear project layout
2	Give it its own Python	Packages from other projects cannot interfere
3	Write down what it needs	Exact packages and versions are recorded, not remembered
4	Keep settings and secrets out of the code	Change a date without editing code, and never commit a key
5	Put it in Git and write a README	There is history, a place to share, and written steps



What we will not do today

> We will not build an interface, we will not measure how good a model is, and we will not call any model. We will only prepare the place where a model will sit later, so setup problems can never block us. Those are Class 3 and later.

> **Check for understanding**
> **Ask:** What does it mean for a program to be reproducible?
> **Answer:** Someone else can follow written steps on their own machine and get the same result.

Transition

> Step one of the map is the simplest: give the code a proper home.

Section 2: Give the code a home

Duration	7 minutes (minute 5 to 12)
Teaching objective	Create a project folder with a clear layout, so every kind of file has one obvious place.

Say

> Right now everything sits in one folder: the script, the data and the output. With one file that is fine. Think of a kitchen where the pans, the spices and the dirty plates all share one drawer. You can cook there for a day, but the moment a second person helps, nobody can find anything.
>
> What do we get from tidying up? Four things, and each one saves real time:
> - **Speed for anyone who opens the project.** A new teammate, or you in three months, finds the data, the code and the results in seconds instead of hunting through one pile.
> - **Fewer accidents.** When raw data, code and output are separate, you don't overwrite a source file by mistake, and you can delete the output folder and rebuild it anytime without fear.

- **Safe sharing.** Later we will put this project on GitHub. With separate folders, we can say “never upload the output or secret files” in one line. In a single pile, that is impossible.
- **Growth without chaos.** Today we have one script. By the end of this class we will have several small files, and in Class 3 more. A layout with a home for each kind of file means adding a file never means rethinking everything.

For Meridian, this matters directly. The ops lead will ask, “Where does this report come from, and which data did it use?” With a clear layout, the answer is a folder name, not a long search through someone’s laptop.

The next steps also depend on it. In Section 3 we give the project its own Python, in Section 4 we install it like a real package, and in Section 5 we split the code into small files. All of that assumes the folders exist. And it is cheap now but painful later: moving files around after fifty exist is a mess.

Here is the layout we are building. Each folder has one job.

```
meridian-workbench/
  src/
    meridian_workbench/     <- the code lives here (a package)
  tests/                    <- checks that prove the rules work
  data/                     <- cases.csv and notes.json
  config/                   <- settings that change between runs
  fixtures/                 <- recorded model responses (used later today)
  outputs/                  <- generated files, never committed
```



Folder	What goes in it
src/meridian_workbench	The program itself, split into small files by job. The folder name is what we import.

Folder	What goes in it
tests	Small programs that check the rules still give the right answers.
data	The raw input files: cases.csv and notes.json.
config	Settings that may change from run to run, such as the batch date.
fixtures	Recorded model responses. They let us practise the AI part with no key and no internet.
outputs	What the program produces, such as attention_report.json. It can be rebuilt at any time, so we will not commit it.

> The `src/` folder might look like an unnecessary extra layer of nesting. So why do we use it? Let's understand this with an everyday example.
>
> Imagine you develop a recipe in your own kitchen. Everything works perfectly because you know where all your ingredients and utensils are. But when you share the recipe with a friend, they struggle to recreate it because the recipe depends on things that were available only in your kitchen.
>
> Code can have a similar problem. If your Python code sits directly in the main project folder, Python finds it simply because you are standing in that folder. Everything works on your laptop even if the install is broken, so you never find out. Your colleague installs it, runs it from somewhere else, and gets a `ModuleNotFoundError`, which means Python cannot find the module it needs.
>
> The `src/` layout removes that accident. Our code sits one level down, inside `src/`, so Python can only find it through the proper install we set up in Section 4. If the setup is wrong, you see the error first, on your own machine, before your colleague does.
>
> Think of it as writing a recipe that includes clear instructions for setting up the kitchen, instead of assuming everyone has your kitchen arranged exactly the way you do.
>
> We'll learn how to configure and install the project properly in Section 4.

Step 1: open new folder in VS Code and add the files

Say

> You used VS Code in Class 1, so you know the window: Explorer on the left, editor in the middle, terminal at the bottom. Today the Explorer earns its place, because it shows our project as a tree and we will build that tree by clicking.
>
> In VS Code choose File > Open Folder. In the dialog, use the New Folder button to create meridian-workbench somewhere sensible, such as Documents, and open it. If VS Code asks whether you trust the folder, click Yes. The Explorer is empty. That is correct: we are building the house from nothing.
>
> Then Terminal > New Terminal, and check that pwd (or Get-Location in PowerShell) shows meridian-workbench. This confirms every command we run next happens inside the project.

Step 2: build the folders

Say

> In the Explorer, click the New Folder icon and create src. Then right-click src, choose New Folder, and create meridian_workbench inside it. Back at the top level, create tests, data, config, fixtures and outputs.
>
> Compare your Explorer with the layout picture. The tree should match. Note the underscore in meridian_workbench: Python cannot import a name with a hyphen.

Step 3: the one small file that makes it a package

Say

Right-click the meridian_workbench folder and choose New File. Name it __init__.py, with two underscores on each side. Open it and type one line, then save:

"""Meridian Case Intake Workbench."""

Why do we need this file? It marks the folder as a regular **package**, a place you can import from. Python can sometimes import a folder without it, but it then treats the folder as a looser "namespace package" that behaves differently and causes confusing problems later. Every proper package has an `__init__.py`, so we always create one. It stays almost empty, because its job is simply to mark the folder.

Two more words go with package, and you will hear all three all class. Picture a recipe binder:
- **Package:** a folder of Python files that Python can import from. You just made one: meridian_workbench. That is the binder.
- **Module:** one Python file inside a package, like one recipe card. The binder has no cards yet. In Section 5 we cut our big script into several.
- **Import:** one file saying "I want to use something from that other file," like writing "see the pancake card" in a recipe. Without it, each file knows only about itself.

We write our first real import in Section 5.

Step 4: copy in the files you already have

- Copy cases.csv and notes.json into the data folder. Copy recorded_responses.json into the fixtures folder. All three are in **Starter Files** if anyone is missing them.
- Copy your finished attention_report.py into the project root for now. In Section 5 we cut it into pieces.

Analogy to use

Recipe binder: the package is the binder, each module is a card.

Common mistakes to expect

- Creating meridian-workbench inside another project folder, or one that is already a Git repository. In the Open Folder dialog, check the path at the top before clicking Open.
- Using a hyphen in the package folder name. Python imports need underscores: meridian_workbench, not meridian-workbench.
- Forgetting __init__.py, or typing it with one underscore. Check the file name in the Explorer before moving on.
- Putting the data files in the project root instead of data/.

If learners struggle

If a learner gets lost in the Explorer, share your screen and compare their tree with the layout picture. The layout matters, not how it was built.

Transition

The folders are ready, so the project has a home. But it will run on the Python already installed on your computer. That Python is shared with every other project you have, and a package you install for this project can break another one. It's like keeping all your ingredients in one shared cupboard with your flatmates: sooner or later, someone's change spoils your dish.

The fix is a private cupboard just for this project. In Python it is called a virtual environment, and building one is our next step.

Section 3: Your own Python: the virtual environment

Duration	10 minutes (minute 12 to 22)
Teaching objective	Create and activate a virtual environment, and prove it is the Python in use.

Say

Your computer has a Python installed. Python can also use extra code that other people have written, such as a tool for reading settings files or checking your spelling. We call each of these add-ons a package. You will meet the tool that fetches them in the next section. For now, one fact matters: when you add a package, it goes into one shared place, inside the Python installed on your computer. Every project you have ever made uses that same place.

That causes three problems:
- **Clashes.** One project needs version 1 of an add-on and another needs version 2. One shared place cannot hold both.
- I**nvisible needs.** Your program may work only because you added something months ago and forgot. Your colleague does not have it, so it fails on their machine. The stranger demo showed the same problem with a data file; with packages it is even harder to spot.
- **No clean undo.** If you add something bad, every project on the laptop is affected.

The fix is a **virtual environment**: a private package space for this project, built on top of the Python installed on your computer. Add-ons go into the project, not into your computer. Now only what this project needs is there, so the list of needs is exact and we can write it down. If it breaks, you delete the folder and rebuild it in a minute.



> **Analogy to use**
> Think of a shared kitchen cupboard in a flat. Everyone puts their ingredients in it. Soon there are two different kinds of flour, someone's old spice spoils your dish, and nobody knows what is theirs. A virtual environment gives each project its own small cupboard, with only what that project needs.

Step 1: check your Python version

macOS or Linux

```
python3 --version
```

Windows PowerShell

```
py --version
```

You need 3.11 or newer. Some tools we use later today need it. Anyone with an older version should stop and ask for help now; do not let it slide.

Step 2: create the environment

macOS or Linux

```
python3 -m venv .venv
```

Windows PowerShell

```
py -m venv .venv
```

Say

> What does python3 -m venv .venv actually say? Read it in three parts:
>
> python3 (or py on Windows) is the Python we are using to do the job.
>
> -m venv means "find the module called venv and run it." A **module** is a piece of Python code you can call by name. venv comes with Python, and it builds private environments. Later today we use the same -m to run pip and even our own project.
>
> .venv is the name we give the new folder. You could call it anything, but .venv is the common convention, so every Python developer recognises it. The leading dot marks it as housekeeping, not part of our project's content.
>
> In plain words: "Python, please run your environment-builder tool and put the result in a folder called .venv."

Run it from inside meridian-workbench. After a few seconds you will see a new folder called .venv. Ask learners to open it and look. It holds a private Python and an empty place for packages. We never edit anything inside it.

Say

> Open the .venv folder and look. It contains a small Python that points back to the installed one, and its own empty packages folder. So a venv does not copy Python and does not change your Python version: it uses the version that created it. What it gives you is a private shelf for packages. (Checking the Python version is the job of the requires-python line we write in Section 4.) Two rules for this folder. We never edit inside it, because it is generated. And we never share it with a colleague, because it contains paths that only exist on your machine, so it would not work on theirs anyway. They build their own in a minute, from a list we will write in Section 4. (In Section 9 we start saving our work with a tool called Git, and there we will make sure this folder is left out.)

Step 3: activate it

macOS or Linux

```
source .venv/bin/activate
```

Windows PowerShell

```
.venv\Scripts\Activate.ps1
```

Creating .venv built the toolbox. It did not pick it up. Until you activate, your terminal still uses the shared Python. So what does activation actually do?

When you type python, the terminal has to find a program with that name. It searches a list of folders, in order. That list is called PATH. Activation puts the .venv's own bin folder (Scripts on Windows) at the front of that list. Now when you type python, the terminal finds the private one first. That is all activation is: it changes which Python answers when you type python. The (.venv) in your prompt is a reminder that this is on.

Let us prove it. Run the first command before activating and the second after.

macOS or Linux

```
which python3
source .venv/bin/activate
which python3
```

Windows PowerShell

```
(Get-Command python).Source
.venv\Scripts\Activate.ps1
(Get-Command python).Source
```

> The second path ends in .venv. The first does not. Same word, different program. (If Windows prints nothing or a WindowsApps path before activating, that is fine: it is still not the .venv one.)
>
> Why every new terminal? The change to PATH lives only inside that one terminal window. A new window starts fresh with the original list. That is why activation is a habit at the start of each session.
>
> Why bother, when we could type the full path each time? We can (it is our fallback for stuck learners). Activation is just a shortcut so we can type `python` and trust it.

> **Platform note: Windows activation**
> If PowerShell says "running scripts is disabled on this system", run this once, then try activating again:
> `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
> On Windows Command Prompt (not PowerShell) the command is .venv\Scripts\activate.bat.



> Look at your prompt. It now starts with (.venv). That is the signal. The name in brackets is the environment you are inside. Every command you run in this terminal now uses the private Python.

Step 4: prove it, do not assume it

```
python -c "import sys; print(sys.prefix)"
```

Why check when the prompt already shows (.venv)? Because the prompt can say (.venv) while another Python answers, especially after switching windows or editors. Printing the path takes one second and removes the guessing.

What Python prints

```
/home/you/meridian-workbench/.venv
```

The path ends in .venv inside your project folder. On Windows it looks like C:\Users\you\meridian-workbench\.venv. If it points anywhere else, the environment is not active.

Step 5: leave and re-enter

```
deactivate
```

The (.venv) tag disappears. Activate again with the same command as Step 3. You will do this at the start of every working session, and every new terminal needs it once.

What each piece does

- Activation changes the terminal so the word python means the project's environment. Nothing is added to the environment by activating.
- sys.prefix is the folder of the environment the running Python is using; inside a venv it is the .venv folder. It is the cross-platform way to prove which environment you are in.
- deactivate undoes activation for that terminal only.

Pause and predict

Ask: "If I open a brand new terminal tomorrow, will (.venv) still be there?" Answer: no. Each terminal must activate once. VS Code can do this for you, as we will see in Section 6.

Check for understanding

Ask: What does the (.venv) at the start of the prompt tell you?

Answer: The terminal is using the project's private Python, so anything we add goes into the project.

Ask: Does deleting the .venv folder delete our code?

Answer: No. The environment is rebuildable and holds no code of ours. That is why we never share it: a colleague builds their own.

Common mistakes to expect

- Running python -m venv .venv from the wrong folder, so .venv lands somewhere else. Check with ls (Get-ChildItem).
- Forgetting to activate. The prompt has no (.venv), so the next package we add in Section 4 lands in the shared Python (see Quiz 1).
- Using the wrong activation command for the operating system or shell.
- Using an old Python (3.9 or 3.10). tomllib will not import later. Install 3.11 or newer.
- Typing deactivate and then wondering why packages vanish. They did not; the terminal simply left the environment.

If learners struggle

If a learner cannot get activation working in the time, let them continue using the full path to .venv/bin/python (macOS/Linux) or .venv\Scripts\python.exe (Windows) for the commands in Section 4. It is clumsy but valid, and you can fix their activation during the lab time.

Right after python -m venv .venv, VS Code may show a pop-up: “We noticed a new environment has been created. Do you want to select it?” Tell learners to close this pop-up for now. We choose the interpreter deliberately in Section 6.

Transition

Now we have a private Python with nothing in it. Next, we choose exactly what goes in, and write it down.

Section 4: pip, pinned dependencies and pyproject.toml

Duration	15 minutes (minute 22 to 37), including Quiz 1
Teaching objective	Install a package into the project, record exact versions, read a pyproject.toml, and install the project itself as an editable package.

Say

Remember the add-ons from the last section? Their proper name is packages, and this section is where we finally add some. A package is code someone else wrote that we reuse, so we do not have to build everything ourselves. pip is the tool that fetches a package from the internet and installs it.

Where does pip fetch from? A website called PyPI (the Python Package Index), a free online library of Python packages. Think of an app store for code. pip downloads from PyPI and installs into whichever Python is running pip. That is why activation matters: it decides where the package lands.

> Today we need two packages. python-dotenv will read a local settings file for us later in the class. ruff will check our code for mistakes. Let us install the first one now, with the environment active.

What is pip?

- **pip** is the tool that downloads and installs packages into Python. The name stands for “Pip Installs Packages”.
- A **package** is code someone else already wrote and tested, so you reuse it instead of writing it yourself.
- pip gets packages from **PyPI**, a free online store for Python code.
- pip installs into **one particular Python**, the one that runs it. That is why we activate `.venv` first.
- Example: `python -m pip install python-dotenv==1.0.1` means “fetch this package, this exact version, and put it in my project’s Python”.

What is ruff, and why do we need it?

Say

> ruff is a code checker. It reads your Python files without running them and points out small problems, such as an import you never use, a variable that is never defined, or a line that is too long. It can also tidy the layout of your code automatically.

- Think of it as a proofreader for code. A proofreader does not write your essay, but they catch the typo you stopped seeing.
- We use it because small mistakes are easy to miss when you read your own code. A tool catches them in a second, every time.
- It also keeps everyone’s code looking the same. When a teammate opens your files, they are not distracted by different spacing or messy imports.
- We will install it in this section and use it in Section 6. For now, just know that it is a helper that checks our work.

Step 1: install a package

```
python -m pip install python-dotenv==1.0.1
python -m pip list
```

What each piece does

- python -m pip means "use the pip that belongs to this Python". It is safer than typing pip alone, because it cannot pick a different Python by accident.
- python-dotenv is the package name. ==1.0.1 pins the exact version. Two equals signs mean "exactly this one".
- pip list shows what is installed. Ask learners what they see: python-dotenv, pip, and not much else. A clean room.

> Why pin the version? Because without a number, pip installs the newest version on the day you run it. A colleague installing a month later may get a different one, and something that worked will quietly stop. Pinning the packages we list is step one. Packages can depend on other packages, and real projects pin those too, with a full list or a lock file. Our two packages depend on nothing else, so today our two pins are the whole story.



Step 2: write pyproject.toml

> So far, if a colleague wants to run our project, we would have to tell them everything by message: “You need Python 3.11. Install python-dotenv. Use version 1.0.1. Also, our code is in the src folder.” They will forget half of it, and so will we. The fix is to write it all down once, in one file that both people and tools can read. That file is called pyproject.toml.

Think of it as the ID card and the instruction sheet of the project. It answers four questions:

- **What is this project called?** The name and version, so tools know what they are installing.
- **Which Python does it need?** If someone has Python 3.9, the file says “stop, you need 3.11 or newer” before anything breaks halfway through.
- **Which packages does it need?** The exact list, with versions, so everyone installs the same things.
- **Where is the code, and how should it be checked?** The settings for tools like ruff live here too, so one file holds everything.

> Without this file, setting up the project depends on someone’s memory. With it, a single command can read the file and build the whole setup. That is what makes the project reproducible. Create pyproject.toml in the project root and paste the file below. We will read it together, block by block.

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

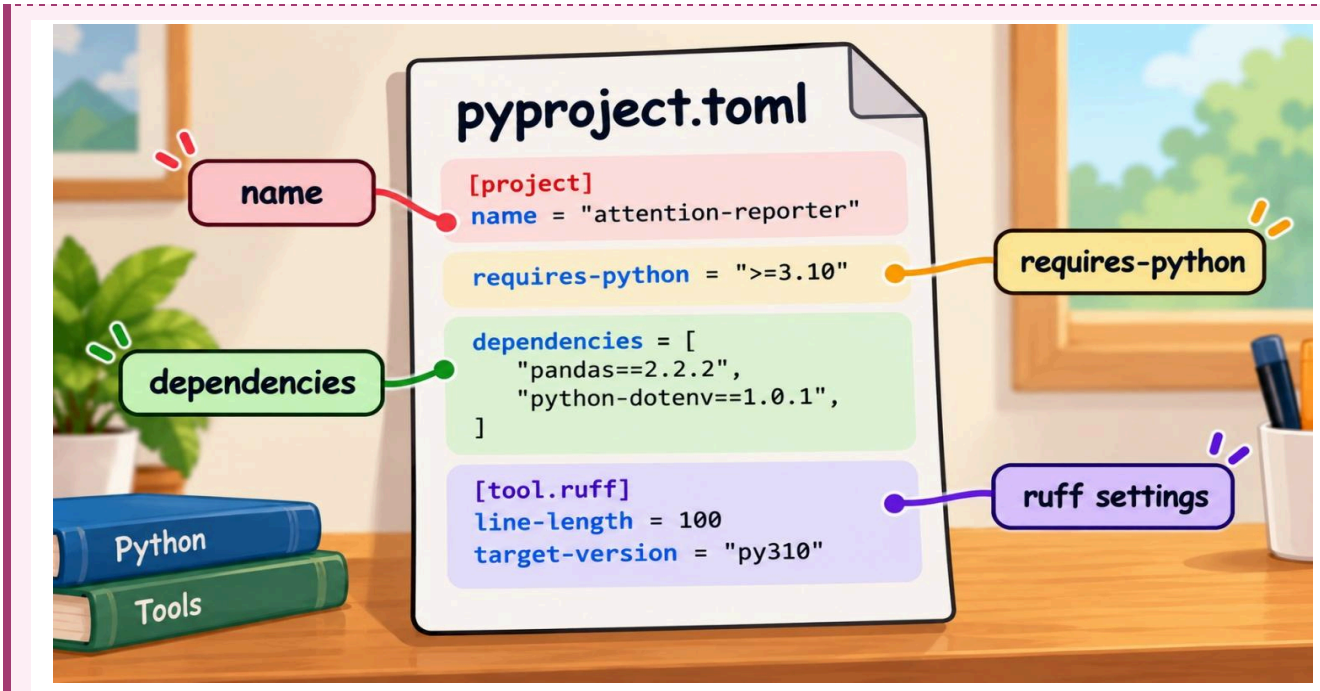
[project]
name = "meridian-workbench"
version = "0.1.0"
description = "Meridian Case Intake Workbench: which cases need attention, with evidence."
requires-python = ">=3.11"
dependencies = ["python-dotenv==1.0.1"]
```

```
[project.optional-dependencies]
dev = ["ruff==0.6.9"]

[tool.setuptools.packages.find]
where = ["src"]

[tool.ruff]
line-length = 100
src = ["src", "tests"]

[tool.ruff.lint]
select = ["E", "F", "I"]
```



Reading the file, block by block

Block	What it says
[build-system]	How to build the project. We use setuptools, a standard tool. Copy it as it is for now.
[project]	Who we are. name is the package name (hyphens allowed here). requires-python is the oldest Python we support. dependencies are the packages the program needs to run.
[project.optional-dependencies]	Extra packages only developers need. ruff goes under "dev", because the program runs without it. requirements.txt pulls it in with [dev].
[tool.setuptools.packages.find]	Tells the build tool that our code lives in src/. This is why the src layout works.
[tool.ruff] and [tool.ruff.lint]	Settings for ruff, the checker. Lines up to 100 characters. The rules E (style errors), F (unused or undefined names) and I (import order).

> TOML is just a settings format. Square brackets name a section, and key = value lines sit inside. It is deliberately boring. You will meet it in every Python project.

Step 3: write requirements.txt

Create requirements.txt in the project root with exactly this one line.

```
-e .[dev]
```

Step 4: install everything from the file

```
python -m pip install -r requirements.txt
```

What Python prints (the last line)

```
Successfully installed meridian-workbench-0.1.0 ruff-0.6.9
```

python-dotenv is not in that line because you already installed it in Step 1. pip only lists what it installed just now. (The exact versions match because the pins are the same.)

What each piece does

- -r requirements.txt means "install everything listed in this file". One command rebuilds the environment for anyone.
- -e .[dev] means "install the project in this folder in editable mode, plus its dev extras". Editable means Python does not copy our code. It adds a pointer to src/, so the import works from anywhere and edits to our code take effect immediately, with no reinstall. (Only a change to pyproject.toml itself, such as a new dependency, needs the install run again.)
- [dev] means "also install the extra group called dev", which is where ruff is pinned in pyproject.toml. So every version number lives in one place, pyproject.toml, and the two files cannot drift apart. requirements.txt is just the one-line install for a stranger.

Say

> A fair question: both files list packages, so why two?
>
> `pyproject.toml` describes the **project**: its name, the Python it needs, and what it depends on.
>
> requirements.txt is the one-line setup: a stranger types a single command and gets everything, including our project itself and the dev tool.
>
> In a small project, requirements.txt can be this short. In a big one, it often grows into the exact full list of every package and version, for rebuilding the same environment.

Step 5: prove the project can be imported

```
python -c "import meridian_workbench; print(meridian_workbench.__file__)"
```

You should see a path ending in src/meridian_workbench/__init__.py. That is the small file we typed in Section 2, Step 3. If it were missing, this line would print None instead of a path, because Python would treat the folder as a namespace package. That is the editable install working. Before Step 4, the same command fails with ModuleNotFoundError; try it in a second terminal without activation if you want a quick demo.

> **Common gotcha**
>
> - Do not use pip freeze > requirements.txt as it is here. In an editable install, freeze writes a line like -e /Users/you/meridian-workbench, a path that only exists on your machine, so a stranger cannot use it. (pip freeze --exclude-editable leaves that line out.) In bigger projects, freeze is a standard way to record every exact version. Today our file is one line, so we write it by hand.
> - Editable installs create a src/meridian_workbench.egg-info folder. It is generated, so it will go in .gitignore later.

A word about uv (1 minute, only after the above works)

> You will meet a newer tool called uv that does the work of venv and pip, much faster. It is worth knowing it exists, and we may use it later in the programme. Today we use venv and pip because they ship with Python and every other tool is built on the same ideas. If you understand today, learning uv later takes ten minutes.

Instructor note

Do not demonstrate uv. Mention it once, for context, and move on. Learners who already use it may continue with it, provided the result is the same: an activated environment, the pinned packages and the editable install.

Check for understanding

Ask: What is the difference between ==1.0.1 and writing nothing?

Answer: ==1.0.1 pins the exact version, so everyone gets the same. Writing nothing installs whatever is newest on the day you run it.

Ask: Why do we use -e . ?

Answer: It installs our own project in editable mode, so the import works everywhere and code edits take effect immediately. [dev] also brings in ruff.

Common mistakes to expect

- Skipping activation, so packages install into the global Python (this is Quiz 1).
- Writing = instead of == for the pin, or leaving a space inside the name.
- Running pip install -r requirements.txt from the wrong folder. The file is not found.
- A missing or misnamed src/meridian_workbench folder, so -e . finds nothing to install or imports fail later.
- Using python3 on Windows or py on macOS inside the activated environment. Once active, plain python is right everywhere.
- On macOS with zsh, typing pip install -e .[dev] directly in the terminal fails with "no matches found". Inside requirements.txt it is fine; on the command line, quote it: pip install -e ".[dev]".

If learners struggle

If -e . fails, read the last line of the error with the learner. It is usually a typo in pyproject.toml (a missing quote or bracket) or a missing src/meridian_workbench folder. You may compare with the final files provided at the beginning.

Quiz 1 of 6 | After Section 4 (minute 37) | Multiple choice (one answer) | about 90 seconds

Checks: Why activation matters before installing.

Question. You forgot to activate .venv, then ran this command in the project folder. Where did the package go?

```
python -m pip install python-dotenv==1.0.1
```

A. Into .venv anyway, because the folder exists in the project

B. Into your computer's global Python, not into .venv

C. Nowhere. pip refuses to run without activation

D. Into the src folder next to the code

Correct answer: B

pip installs into whichever Python runs it. Without activation, that is the global Python, so .venv never receives the package and other projects can break.

A is a very common belief: a folder named .venv does nothing until it is activated. C and D describe behaviour pip does not have.

Instructor tip: Some systems (newer Linux and Homebrew Python) refuse with "externally-managed-environment". Use that as a free demo of the same lesson.

Transition

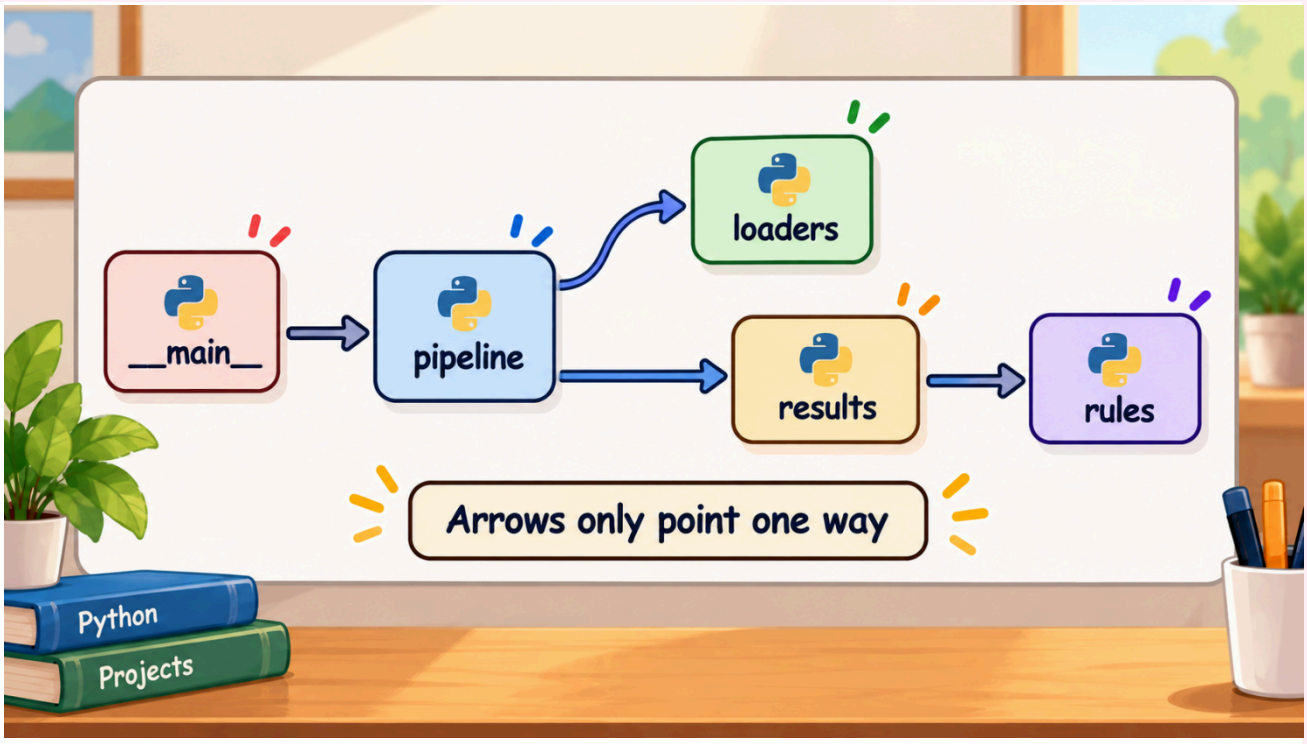
> Now that the project installs, we can safely cut the Class 1 script into pieces, because every piece can find the others.

Section 5: Split the script into modules

Duration	13 minutes (minute 37 to 50), including Quiz 2
Teaching objective	Cut the single Class 1 file into small modules with one job each, and run the program with python -m.

Say

> Our attention_report.py does five jobs in one file: read files, apply the rule, build a result, run the whole thing and check itself. When someone wants to change just the rule, they have to read everything. We will split it so that each file has one job and a name that says what it is.
>
> Remember the binder from Section 2. Each new file is a module inside the package meridian_workbench. Now the import becomes real: another file will write from meridian_workbench.rules import needs_attention to use a function from this one.
>
> Each of these files is a module. What we are doing has a name: refactoring, meaning changing how code is organised without changing what it does. That is why we run it before and after and compare output. If the output is identical, the move was safe.



> Look at the arrows. pipeline uses loaders and results. results uses rules. Nothing ever points back. If rules read files, or loaders used the rules, the arrows would tangle and every change would risk breaking something unrelated. One job, one direction.

File in src/meridian_workbench	Its one job
loaders.py	Read cases.csv and notes.json and hand back Python data. Nothing else.
rules.py	The decision: is this case closed, and how many days overdue is it? Pure logic, no files.
results.py	Combine a case, its note and the rule outcome into one evidence-backed result.
pipeline.py	Run the steps in order: load, decide, save, report. This is our main().
__main__.py	A tiny file that makes python -m meridian_workbench work.

Step 1: create the four modules

In src/meridian_workbench, create each file below. Learners cut the matching functions from attention_report.py and paste them in, then add the import lines at the top. Full text is shown so they can compare.

loaders.py

```
"""Read the raw case files."""

import csv
import json

def load_cases(path):
    with open(path, newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))

def load_notes(path):
    with open(path, encoding="utf-8") as f:
        return json.load(f)
```

rules.py

```
"""Deterministic attention rules. Python decides this, never the model."""

from datetime import date

CLOSED_STATUSES = {"DELIVERED", "CANCELLED"}

def days_overdue(promised_date_text, as_of):
    """Whole days between the promised date and the batch date."""
    promised = date.fromisoformat(promised_date_text)
    return (as_of - promised).days

def needs_attention(case, as_of):
    """Return (flag, reason)."""
    if case["order_status"] in CLOSED_STATUSES:
        return False, "CLOSED"
    if days_overdue(case["promised_date"], as_of) > 0:
        return True, "OVERDUE_NOT_CLOSED"
    return False, "ON_TIME"
```

Say

> Deterministic means the same input always gives the same answer. Pure means the function uses only what you hand it, with no files, no internet and no hidden state. Rules should be both. That is why we can test them with four made-up cases and no data files.

results.py

```
"""Build one evidence-backed result per case."""

from meridian_workbench.rules import days_overdue, needs_attention


def build_result(case, note, as_of):
    flag, reason = needs_attention(case, as_of)

    days = 0
    if flag:
        days = days_overdue(case["promised_date"], as_of)

    note_id = None
    if note:
        note_id = note["note_id"]

    if flag:
        evidence = (
            f"Status {case['order_status']}, promised {case['promised_date']}, "
            f"{days} days overdue as of {as_of.isoformat()}"
        )
    else:
        evidence = (
            f"Not flagged ({reason}). Status {case['order_status']}, "
            f"promised {case['promised_date']}"
        )

    return {
        "case_id": case["case_id"],
        "order_id": case["order_id"],
        "order_status": case["order_status"],
        "promised_date": case["promised_date"],
        "batch_date": as_of.isoformat(),
        "needs_attention": flag,
        "reason": reason,
        "days_overdue": days,
        "evidence": evidence,
        "source_note_id": note_id,
        "needs_interpretation": flag and note is not None,
    }
```

pipeline.py

```
"""Run the Meridian attention report from start to finish."""

import csv
import json
from datetime import date
from pathlib import Path

from meridian_workbench.loaders import load_cases, load_notes
from meridian_workbench.results import build_result

DATA_DIR = Path("data")
OUTPUT_DIR = Path("outputs")
BATCH_DATE = date(2025, 3, 10)


def main():
    cases = load_cases(DATA_DIR / "cases.csv")
    notes = load_notes(DATA_DIR / "notes.json")

    notes_by_case = {}
```

```
    for note in notes:
        notes_by_case[note["case_id"]] = note

    results = []
    case_ids = []
    for case in cases:
        note = notes_by_case.get(case["case_id"])
        results.append(build_result(case, note, BATCH_DATE))
        case_ids.append(case["case_id"])

    if len(results) != len(cases):
        raise ValueError("Every case must produce exactly one result.")
    if len(set(case_ids)) != len(case_ids):
        raise ValueError("Case IDs must be unique.")

    OUTPUT_DIR.mkdir(exist_ok=True)
    with open(OUTPUT_DIR / "attention_report.json", "w", encoding="utf-8") as f:
        json.dump(results, f, indent=2)

    flagged_count = 0
    for r in results:
        if r["needs_attention"]:
            flagged_count += 1
            print(f"{r['case_id']}: {r['reason']} ({r['days_overdue']} days)")
    print(f"{flagged_count} of {len(results)} cases need attention")


if __name__ == "__main__":
    main()
```

__main__.py

```
"""Allow: python -m meridian_workbench"""

from meridian_workbench.pipeline import main

if __name__ == "__main__":
    main()
```

What each piece does

- from meridian_workbench.rules import ... is an absolute import. It names the package, so it works the same wherever the file is used. This is what the editable install from Section 4 makes possible.
- pipeline.py holds the old main() almost unchanged. Only the top changed: the paths and the batch date are now named constants, and the work calls the other modules. We will move those constants into a settings file in Section 7.
- The two safety checks at the end of main() now raise a ValueError instead of using assert. assert is for tests (Section 8). Python can be started with a flag (-O) that skips every assert, so real programs raise real errors.
- __main__.py is a special filename. When you run python -m meridian_workbench, Python runs this file. It imports main and starts it behind the same guard from Class 1 Section 9b. (Quick reminder: if __name__ == "__main__": means “run this only when the file is started directly, not when another file imports it”. Without it, merely importing pipeline would start the whole report.)
- CLOSED_STATUSES stays in rules.py. It is a business rule, not a setting. We come back to that distinction in Section 7.

The self-check function from Class 1 (run_self_checks) is not moved into any module. We rebuild it properly as a test in Section 8. Park attention_report.py outside the project folder or delete it once the new run works.

Step 2: run the program as a module

From the project root with .venv active:

```
python -m meridian_workbench
```

What Python prints

```
C-1002: OVERDUE_NOT_CLOSED (3 days)
C-1004: OVERDUE_NOT_CLOSED (2 days)
C-1005: OVERDUE_NOT_CLOSED (4 days)
C-1006: OVERDUE_NOT_CLOSED (2 days)
4 of 7 cases need attention
```

Pause and predict

Before you run it, ask: "We moved everything around. Do you expect the output to change?" It should not. Same input, same rule, same answer. A refactor changes the shape, not the result.

Same four cases, same days, same count. That is how you know the split is safe: the output did not move. Notice that python -m means "run this installed package", not "run this file".

`python somefile.py` runs one file, and that file does not know it belongs to a package. `python -m meridian_workbench` says “find the installed package by name and run it”. That is the same lookup any other program will use. It also needs `__main__.py` because that is the file Python looks for to start a package.

It only works because of Section 4.

You may have noticed that pipeline.py still imports csv, which is now used only in loaders.py. Leave it alone. Python does not mind. In the next section a tool will find it for us.

Analogy to use

A kitchen brigade. One person chops, one cooks, one plates. Each can be replaced without retraining the others, and nobody needs to know how the sauce is made to plate the dish.

Check for understanding

Ask: Why does rules.py not read files?

Answer: One job per file. If the rule only takes data in and gives an answer out, we can test it without any files and reuse it anywhere.

Ask: What does python -m meridian_workbench do?

Answer: It runs the installed package through __main__.py, which starts main.

Common mistakes to expect

- Running python -m meridian_workbench from a different folder: the relative paths ("data") will not be found. We fix this in Section 7.
- Forgetting to change imports after moving a function: NameError or ImportError. Read the last line of the error.
- Circular imports: rules.py importing from results.py while results.py imports from rules.py. Arrows only point one way.
- Naming the package folder with a hyphen, or forgetting __init__.py.

- Moving code and also changing it. Move first, run, then change.

Quiz 2 of 6 | After Section 5 (minute 50) | Multiple Choice Question | about 90 seconds

Checks: Editable install and one job per module.

Question. The Workbench is split into modules and installed with pip install -e .

A. After every edit to rules.py you must run pip install again
B. rules.py should import loaders so it can read the cases itself
C. While .venv is active, import meridian_workbench works from any folder
D. rules.py should also contain the code that reads the case files

Correct answers: C

C is true because the editable install points Python at src/, so the import works wherever you are. Edits show up straight away, so A is false.

B and D are false for the same reason. Each module has one job and arrows only point one way, so rules.py holds rule logic only and never reads files. Mixing file reading into the rules makes them harder to test.

Instructor tip: Ask learners to say how many answers are correct before they pick. Tell them "two".

Transition

> The code is tidy. Now let us make sure the tools around it keep it tidy, starting with telling VS Code which Python to use.

Section 6: VS Code interpreter and ruff

Duration	7 minutes (minute 50 to 57)
Teaching objective	Point VS Code at the project's Python and use ruff to find and fix small mistakes.

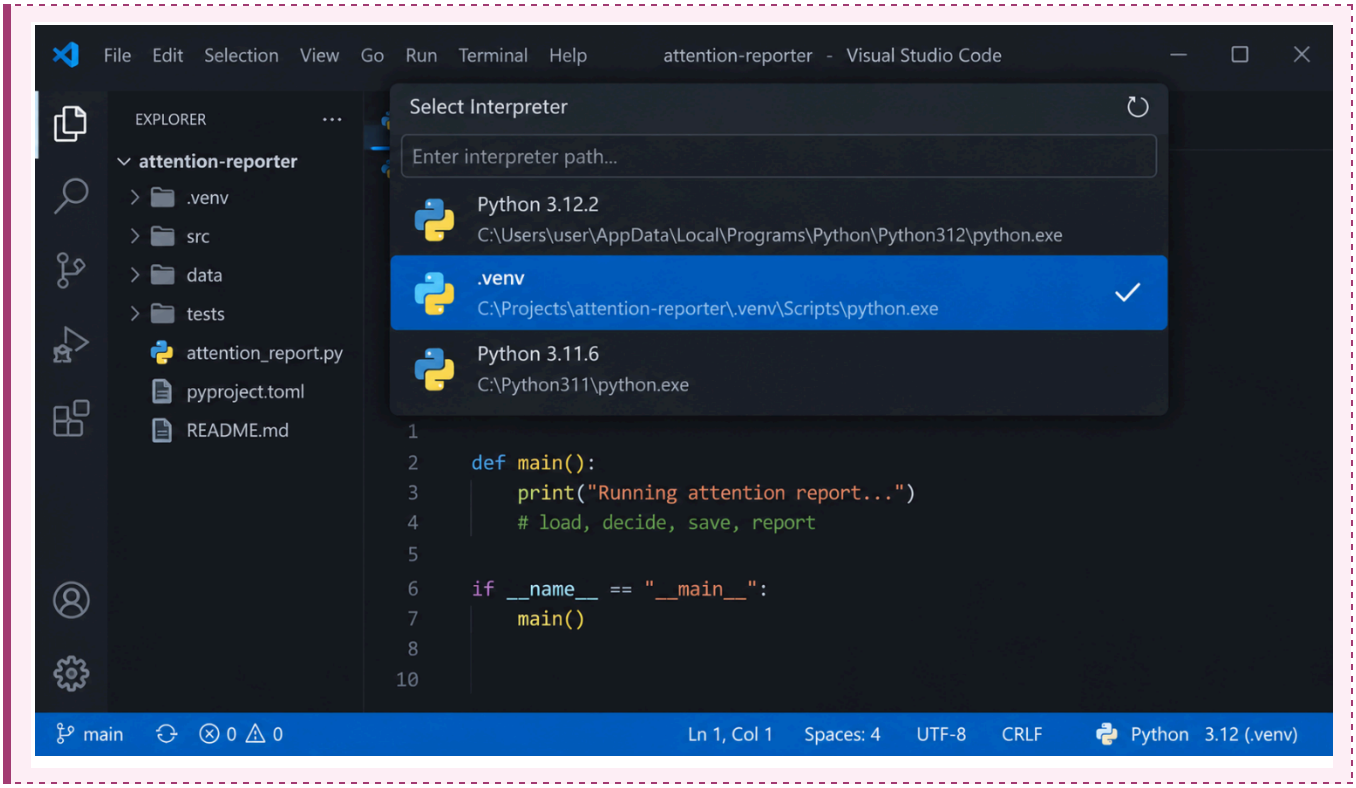
Say

> VS Code is a separate program from your terminal. It does not know about our .venv unless we tell it. If it uses a different Python, the editor will underline imports that are fine, and the Run button will use the wrong Python. This takes ten seconds to fix.
>
> You already saw the status bar in Class 1, where it showed which Python version VS Code was using. Today there are two Pythons to choose from: the shared one and the one inside .venv.
>
> Your code is just text. Something has to read it and carry it out. That something is the **interpreter**, which is simply the `python` program. A computer can have several: the shared one and one inside each `.venv`. VS Code cannot guess which you mean. If it picks the wrong one, it cannot see your installed packages, so it underlines `import dotenv` in red even though the code runs fine in the terminal. Selecting the interpreter tells VS Code “use the Python inside `.venv`”.

Step 1: select the interpreter

- Press Ctrl+Shift+P (Cmd+Shift+P on macOS) to open the command palette.
- Type Python: Select Interpreter and press Enter.
- Choose the entry that shows .venv in its path, usually marked "Recommended".
- Look at the bottom-right status bar. It should say the Python version and (.venv).
- Open a new terminal in VS Code (Terminal > New Terminal). VS Code activates the environment for you. Depending on your VS Code version you may or may not see (.venv) in the prompt, so check with the Section 3 command: python -c "import sys; print(sys.prefix)".

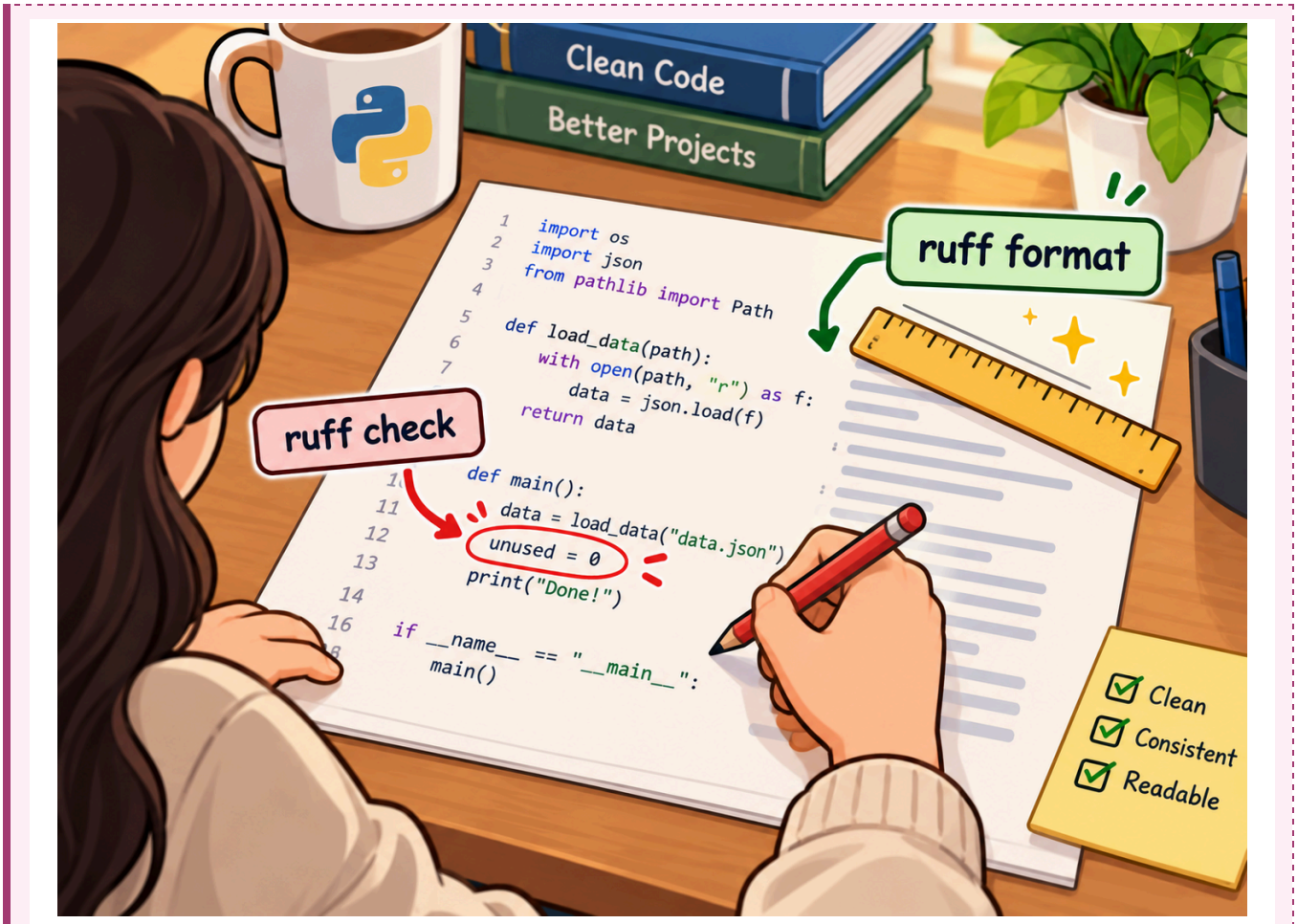
- The command palette is VS Code's search box for commands. The status bar is the strip at the bottom that you met in Class 1.



> The interpreter is the Python that runs your code. Choosing the one inside .venv means the editor, the terminal and our own commands all agree. This is the single most common reason for "it works in the terminal but VS Code shows an error".

Step 2: meet ruff

> Remember ruff from Section 4, our code proofreader? Now we use it. ruff is a fast checker. It reads our code without running it, and tells us about small mistakes, such as imports we never use, lines that are too long, and imports in the wrong order. It is like a proofreader with a red pencil.
>
> Two different jobs. A linter looks for likely mistakes: that is `ruff check`. A formatter only rearranges spacing and layout: that is `ruff format`. ruff does both. Why use a formatter? So nobody on a team spends review time arguing about spacing.




```
ruff check .
```

What ruff prints

```
src/meridian_workbench/pipeline.py:3:8: F401 [*] `csv` imported but unused
Found 1 error.
[*] 1 fixable with the `--fix` option.
```

That is the leftover import csv from Section 5. ruff also prints the offending line itself, with a ^ marker under csv; we show only the summary lines here. ruff caught it without us noticing. Now let it fix it:

```
ruff check --fix .
ruff format .
ruff check .
```

What each piece does

- ruff check . looks at every Python file below this folder. The dot means "here".
- --fix lets ruff repair what it can repair safely. Deleting an unused import is safe. Always read what it changed.
- ruff format . tidies spacing and layout to a single standard style. Do not argue about style in a team: let the tool decide.
- The last ruff check . should print All checks passed!

- The settings (line length 100, rules E, F, I) come from the [tool.ruff] blocks in pyproject.toml, the file we read in Section 4.

> One more thing we are not doing today: type checking. A second tool, mypy, can check that the right kind of value is passed to each function. That arrives in Class 3, when we have functions that deserve it.

Instructor note

ruff format should report that the files are left unchanged, because the code we pasted is already in its style. On learners' own edits it will tidy spacing and blank lines; that is expected and harmless. Do not install the VS Code ruff extension in class; it creates one more thing to debug. Mention it as optional at the end.

Check for understanding

Ask: Does ruff run our program?

Answer: No. It only reads the code and reports problems.

Ask: Why select the interpreter in VS Code?

Answer: So the editor, the Run button and the terminal all use the project's Python, not some other one.

Common mistakes to expect

- Choosing the global Python in the interpreter list, so imports are underlined.
- Running ruff without an activated environment: command not found. Activate first, or call it by path: .venv/bin/ruff (macOS/Linux) or .venv\Scripts\ruff.exe (Windows).
- Running ruff check . outside the project folder.
- Accepting every --fix without looking. Always look at the diff.

If learners struggle

If .venv does not appear in the interpreter list, choose "Enter interpreter path", then Find, and select .venv/bin/python (or .venv\Scripts\python.exe). Then reload the window.

Transition

> The code is tidy and the tools agree. Now to a problem the whole class has: the batch date and folder paths are written inside the code. Anyone who wants a different date has to edit the program.

Section 7: Settings, environment variables and secrets

Duration	13 minutes (minute 57 to 70), including Quiz 3
Teaching objective	Separate rules from settings from secrets; read settings from a file; override with an environment variable; keep local files and secrets out of Git.

Say

> Right now the batch date is written in pipeline.py. If the ops lead wants to run for the 13th, someone must edit code. If we later need a password or an API key, it would end up in code too, and then in Git, and then on the internet. Let us sort the things a program knows into three bins.



Bin	What goes in it	Example in the Workbench
Rules	Business logic the company decided. Changes need review and a test. Lives in code.	CLOSED_STATUSES: DELIVERED and CANCELLED are closed, no matter who runs it.
Settings	Values that change between runs or machines. Not secret. Lives in a file.	The batch date, the data folder, the output folder, the AI mode.
Secrets	Anything that gives access. Never in code, never in Git. Lives in the environment.	An API key (we have none yet, but we will prepare the place).

Say

> I said secrets live "in the environment". Let me define that. An **environment variable** is a named value that your terminal holds in memory and hands to any program started from it. It is not stored in any file of your project. So Git cannot see it, and it cannot be committed by accident. Setting one changes nothing in your code.
>
> A fair question: "But `.env` is a file in my project." Yes. It is a local file that **loads** values into the environment for you so you do not retype them. It is safe only because `.gitignore` keeps it out of Git. That is why Step 6 comes right after Step 5.
>
> **Why not put secrets in `settings.toml`?** Because `settings.toml` is committed. Anything in it is public to every reader, forever.
>
> An API is a way for one program to ask another program to do something, for example our program asking an AI provider to read a note. An API key is the password that proves you are allowed to ask, and it is usually what the provider bills. Anyone who has your key can spend your money.

Pause and predict

Ask: "Is the list of closed statuses a setting?" Take answers. The right answer is no: it is a business rule. If a setting changes it quietly, the meaning of "needs attention" changes without a code review. Keep rules in code.

Step 1: create config/settings.toml

```
# Settings change between runs or machines. Business rules do not live here.

[paths]
data_dir = "data"
output_dir = "outputs"
fixtures_dir = "fixtures"

[rules]
batch_date = "2025-03-10"
```

Step 2: create src/meridian_workbench/config.py

```
"""Load settings from config/settings.toml. Environment variables override the file."""

import os
import tomllib
from datetime import date
from pathlib import Path

from dotenv import load_dotenv

PROJECT_ROOT = Path(__file__).resolve().parents[2]
SETTINGS_FILE = PROJECT_ROOT / "config" / "settings.toml"


def load_settings():
    """Return the settings as a dictionary."""
    load_dotenv(PROJECT_ROOT / ".env")

    with open(SETTINGS_FILE, "rb") as f:
        raw = tomllib.load(f)

    batch_date_text = os.environ.get("MERIDIAN_BATCH_DATE", raw["rules"]["batch_date"])

    return {
        "data_dir": PROJECT_ROOT / raw["paths"]["data_dir"],
        "output_dir": PROJECT_ROOT / raw["paths"]["output_dir"],
        "fixtures_dir": PROJECT_ROOT / raw["paths"]["fixtures_dir"],
        "batch_date": date.fromisoformat(batch_date_text),
    }
```

What each piece does

- Path(__file__).resolve().parents[2] finds the project root from the location of this file: config.py is in src/meridian_workbench, so two folders up is the project. Because it starts from the file and not from where you typed the command, the program works from any folder. This works because we installed with -e ., so Python runs config.py from inside our project folder. Packaging data and settings for a normal install is a later topic.
- open(..., "rb") opens the file as bytes. tomllib requires "rb". With "r" it raises a TypeError, which is a classic first error.
- os.environ.get("NAME", default) reads an environment variable and falls back to the file value if it is not set. This line is the whole override mechanism.
- date.fromisoformat turns the text "2025-03-10" into a real date, as in Class 1.

- load_dotenv reads a local .env file, if there is one, and adds its values to the environment. We will meet it in a moment.

Step 3: make pipeline.py use the settings

Replace the three constants and the three places that used them. Add the new import **above** the loaders import, so the three imports stay in alphabetical order (config, loaders, results). The edited parts of pipeline.py:

```
from meridian_workbench.config import load_settings
from meridian_workbench.loaders import load_cases, load_notes
from meridian_workbench.results import build_result


def main():
    settings = load_settings()
    cases = load_cases(settings["data_dir"] / "cases.csv")
    notes = load_notes(settings["data_dir"] / "notes.json")
    ...
        results.append(build_result(case, note, settings["batch_date"]))
    ...
    settings["output_dir"].mkdir(exist_ok=True)
    with open(settings["output_dir"] / "attention_report.json", "w", encoding="utf-8") as f:
```

Delete DATA_DIR, OUTPUT_DIR and BATCH_DATE at the top. The finished pipeline.py is in the final directory linked at the start of this script. Run python -m meridian_workbench again; the output is the same four cases. Then let ruff tidy up after us:

```
ruff check --fix .
```

What ruff prints

```
Found 3 errors (3 fixed, 0 remaining).
```

Three fixes: the imports of date and Path, which pipeline.py no longer needs, and the spacing of the import block (rule I001), because deleting the constants left the wrong number of blank lines behind. That is the whole point of a checker: it keeps up with our edits. (pipeline now has one new arrow, to config. config imports nothing of ours, so the arrows still point one way.)

> Now a small proof that it works from anywhere. Go to your home folder and run the same command.

macOS or Linux

```
cd ~
python -m meridian_workbench
cd -
```

Windows PowerShell

```
Push-Location ~
python -m meridian_workbench
Pop-Location
```

Because the paths now start from PROJECT_ROOT, the output is the same. Note: the (.venv) is still active, so python still means our Python. If you open a fresh terminal, activate first. (On Windows we use Push-Location and Pop-Location to go there and back, because cd - only works in PowerShell 7 and newer.)

Step 4: override from the environment

> The ops lead wants the report as of the 13th. No code edit. We change the environment for one run.

macOS or Linux

```
MERIDIAN_BATCH_DATE=2025-03-13 python -m meridian_workbench
```

Windows PowerShell

```
$env:MERIDIAN_BATCH_DATE = "2025-03-13"
python -m meridian_workbench
Remove-Item Env:MERIDIAN_BATCH_DATE
```

Say

> On macOS and Linux, putting the variable before the command applies it to that **one run only**. In PowerShell, `$env:NAME = ...` stays set for the whole terminal session. That is why we must remove it afterwards, or the date stays stuck (see the last common mistake in this section).

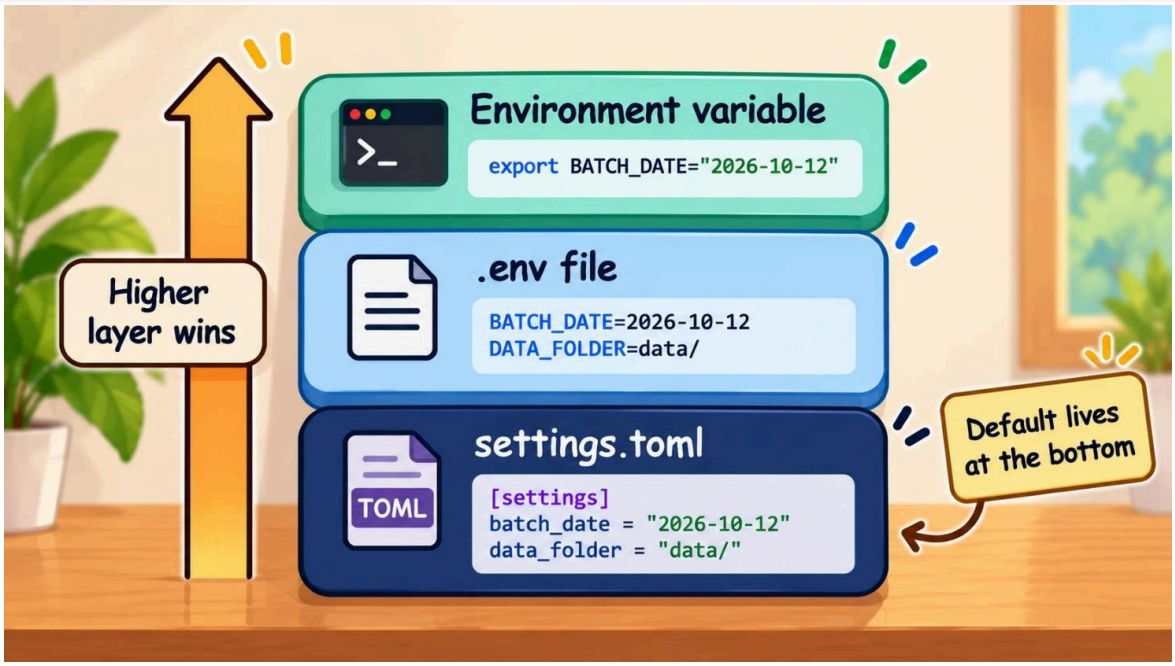
What Python prints

```
C-1002: OVERDUE_NOT_CLOSED (6 days)
C-1003: OVERDUE_NOT_CLOSED (1 days)
C-1004: OVERDUE_NOT_CLOSED (5 days)
C-1005: OVERDUE_NOT_CLOSED (7 days)
C-1006: OVERDUE_NOT_CLOSED (5 days)
5 of 7 cases need attention
```

Pause and predict

Before running: "Why will there be five cases and not four?" Because C-1003 was promised for the 12th, so on the 13th it is one day late. Same rule, a different input.

> Same code, different answer, and nobody edited a file. That is what a setting is. The order of authority is: environment variable first, then the settings file as the default.



Step 5: .env for local convenience, .env.example for the team

Typing a variable every time is tedious. python-dotenv lets us keep local values in a file called .env, which load_dotenv reads. Create .env in the project root with one line, then run again:

```
MERIDIAN_BATCH_DATE=2025-03-13
```

The output now shows five cases without you typing anything. Delete the line again (or comment it with #) to return to the default. Then create .env.example, a template that the team can copy:

```
# Copy this file to .env for local use. Never commit .env.
# Optional overrides. Leave them commented out to use config/settings.toml.
# MERIDIAN_BATCH_DATE=2025-03-10
# MERIDIAN_AI_MODE=recorded

# Placeholder for Class 3. Not used yet. Uncomment and fill in only in your own .env.
# MERIDIAN_LLM_API_KEY=
```

- .env is yours. It holds real values for your machine and is never shared.
- .env.example is the team's. It shows which variables exist, with no real values, and is committed.
- load_dotenv does not override a variable already set in the real environment. A value you typed in the terminal wins over the .env file.

Step 6: .gitignore

> Git is the tool we use in Section 9 to record the history of the project. For now, one word: a file is tracked when Git records its changes. By default Git would track everything in the folder. .gitignore is our list of “never track these”.
>
> Git will track everything in the folder unless we say otherwise. Some files must never be tracked: the environment, the .env file with secrets, generated outputs and caches. A file called .gitignore lists them.

```
# Virtual environment
.venv/

# Secrets and local settings
.env

# Generated files
outputs/

# Python caches and build files
__pycache__/
*.pyc
*.egg-info/
.ruff_cache/

# Editor and system files
.vscode/
.DS_Store
```

What each piece does

- Each line is a pattern. A name ending in / is a folder. .venv/ ignores the whole environment, which anyone can rebuild with requirements.txt.
- *.egg-info/ and __pycache__/ are generated by Python and pip. They are noise in history.
- outputs/ is ignored because the report can be rebuilt by running the program. A committed output would just go stale.
- Lines starting with # are comments.

Common gotcha

- .gitignore only stops files that Git does not yet track. If a secret was ever committed, adding it to .gitignore later does nothing. The secret is already in history. Treat it as compromised and replace it.
- A secret in code is a secret in every clone. "The repository is private" is not a safe plan: access changes and history stays.

Analogy to use

The settings file is the thermostat dial: anyone may turn it. The rules are the building's fire code: you do not change them from the dial. The secrets are the keys on your own keyring: you do not copy them onto the notice board.

Check for understanding

Ask: Where does CLOSED_STATUSES belong, and why?

Answer: In rules.py, because it is a business rule that needs review and tests, not a value that changes per run.

Ask: Which wins: .env or a variable typed in the terminal?

Answer: The variable typed in the terminal. load_dotenv does not override real environment variables.

Common mistakes to expect

- Opening settings.toml with "r" and getting a TypeError. It needs "rb".
- Seeing 2 errors instead of 3 after ruff check --fix. The third is only the blank-line spacing after the imports, so it depends on exactly which blank lines were deleted with the constants. Both are fine. If the config import went below the others, --fix sorts it too.
- Writing the date as 13-03-2025. The format is year-month-day: 2025-03-13.
- Leaving spaces around = in the .env file. python-dotenv tolerates them, but the standard form is NAME=value with no spaces, and a shell rejects spaces if you ever load the file with source.
- Putting real values into .env.example. It is committed and public.
- Creating .gitignore after running git add . and expecting it to unstage files. We create it first.
- Forgetting to remove the environment variable after the demo, then wondering why the date is stuck. Close the terminal or unset it.

If learners struggle

If config.py is too much at once, show only three lines on screen: the file name, the get with a default, and the return. Point them to the full file in the final directory linked at the start of this script and say "we use it, we do not need to write it from memory".

Stretch track (same lab, same time)

Add a new setting called output_file to settings.toml, read it in config.py and use it in pipeline.py, so the output filename comes from the settings file. Then override it with an environment variable.

Quiz 3 of 6 | After Section 7 (minute 70) | Multiple Choice Question | about 90 seconds

Checks: What may be committed to Git.

Question. Which ONE of these is safe to commit to a public GitHub repository?

A. `config/settings.toml` containing the batch date, folder names and the database password

B. `.env` containing a real API key

C. `.env.example` with an empty placeholder for the key

D. `rules.py` with the key typed in, because the repository is private

Correct answers: C

C is safe because a placeholder file holds no real value, and it helps the next engineer see which variables they need to set.

A, B and D all put a secret into Git. Even in a private repository, a key in code travels with every clone, fork and history entry. Treat it as lost and rotate it if it happens.

Instructor tip: Point out that .env.example exists so a stranger can see which variables matter without seeing any real values.

Transition

> We can now change inputs without touching the rules. How do we know the rules themselves are still right? We need proof, and a place for it.

Section 8: A place for proof: tests

Duration	5 minutes (minute 70 to 75)
Teaching objective	Move the Class 1 self-checks into tests/, run them, and break a rule on purpose to see them catch it.

Say

> In Class 1 you wrote small checks at the bottom of the script. They were useful, and they were stuck inside the file. Now that the rules have their own module, the checks get their own folder. When someone changes the rules next month, one command tells them if they broke anything.
>
> A test is a small program that checks another program gives the right answer. Why not just run the whole report and read the output? Because that depends on someone looking carefully every time, and the real data may not contain every case. A test checks the same cases the same way, in a second, and lets us invent cases the real data never has, such as a cancelled order.
>
> The checking word is assert: it says “this must be true”. If true, nothing is printed. If false, the program stops and shows the line.
>
> Why break it on purpose? Because a check that has never failed has not proved it can. Seeing it catch a mistake is what earns our trust in it.

Step 1: create tests/check_rules.py

```
"""Plain assertion checks for the attention rule. pytest arrives in Class 3."""

from datetime import date

from meridian_workbench.rules import days_overdue, needs_attention

BATCH_DATE = date(2025, 3, 10)


def run_checks():
```

```
    closed = {"order_status": "DELIVERED", "promised_date": "2025-03-06"}
    cancelled = {"order_status": "CANCELLED", "promised_date": "2025-03-05"}
    late = {"order_status": "SHIPPED", "promised_date": "2025-03-07"}
    early = {"order_status": "SHIPPED", "promised_date": "2025-03-12"}

    assert needs_attention(closed, BATCH_DATE) == (False, "CLOSED")
    assert needs_attention(cancelled, BATCH_DATE) == (False, "CLOSED")
    assert needs_attention(late, BATCH_DATE) == (True, "OVERDUE_NOT_CLOSED")
    assert needs_attention(early, BATCH_DATE) == (False, "ON_TIME")
    assert days_overdue("2025-03-07", BATCH_DATE) == 3
    print("All rule checks passed")


if __name__ == "__main__":
    run_checks()
```

```
python tests/check_rules.py
```

What Python prints

```
All rule checks passed
```

What each piece does

- assert condition means "this must be true; if not, stop with an error". It is the simplest possible check.
- The four small dictionaries are tiny made-up cases: one closed, one cancelled, one late, one not yet due. Each shows one branch of the rule.
- from meridian_workbench.rules import ... works from the tests folder because of the editable install. It is the same import a colleague will use.
- The if __name__ guard is the one from Class 1 Section 9b: run directly, it checks; imported, it does nothing.

Step 2: break it on purpose

Open rules.py and delete "CANCELLED" from CLOSED_STATUSES so it reads {"DELIVERED"}. Save, and run the checks again.

What Python prints

```
Traceback (most recent call last):
  ...
  File "/home/you/meridian-workbench/tests/check_rules.py", line 17, in run_checks
    assert needs_attention(cancelled, BATCH_DATE) == (False, "CLOSED")
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError
```

> The traceback points at the exact line: the cancelled case. Your path will be longer; look for line 17 and the ^^^ marker. We broke the rule and the check noticed in a second, without anyone reading the code.

Before you put it back, run the report too

```
python -m meridian_workbench
```

What Python prints

```
C-1002: OVERDUE_NOT_CLOSED (3 days)
C-1004: OVERDUE_NOT_CLOSED (2 days)
C-1005: OVERDUE_NOT_CLOSED (4 days)
C-1006: OVERDUE_NOT_CLOSED (2 days)
```

```
C-1007: OVERDUE_NOT_CLOSED (5 days)
5 of 7 cases need attention
```

> No error and no warning. The report happily says 5 of 7, and C-1007, a cancelled order, is now "overdue". Without the check, nobody would notice until a customer did. Now put "CANCELLED" back and run the checks again. That is what proof is for.

> **Instructor note**
> These are plain assertions and not pytest on purpose. pytest, with proper test names and reports, is Class 3. Do not explain pytest here.
> assert belongs in tests like these. In the program itself we raise real errors instead (Section 5), because Python run with -O skips every assert.

> **Check for understanding**
> **Ask:** What does assert do when its condition is false?
> **Answer:** It stops the program with an AssertionError that points at the line.

> **Common mistakes to expect**
> - Forgetting to restore CANCELLED after the demo.
> - Running the test from a terminal where .venv is not active: ModuleNotFoundError (see Quiz 5).
> - Writing = instead of == inside an assert. Python reports a SyntaxError.

> **If learners struggle**
> If time is short, show the break-and-catch live with learners watching, and let them type the file during the lab.

Transition

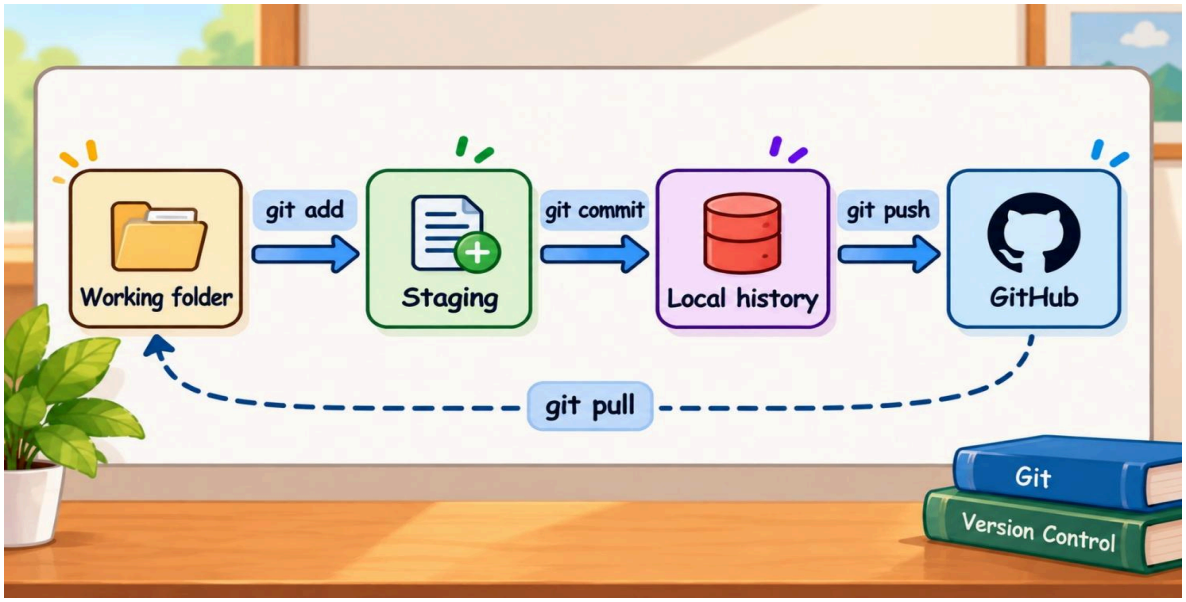
> We have a tidy, configurable, tested project on one laptop. A repository is what turns it into something others can use. Time for Git.

Section 9: Git basics

Duration	18 minutes (minute 75 to 93), including Quiz 4
Teaching objective	Create a repository, make a first commit, publish it to GitHub, then clone and pull to see how a second person receives changes.

Say

> Without a tool, people save `report_final.py`, `report_final2.py`, `report_REALLYfinal.py`. Nobody knows what changed, who changed it, or which is right, and two people editing the same file overwrite each other. **Version control** is a system that records each saved state of a project with who made it and why, so you can compare, go back and work as a team. Git is the most widely used one. A **repository** (repo) is a project folder that Git is tracking, along with its full history. Git is a tool that remembers every saved version of the project, who changed what, and why. GitHub is a website that stores a copy so other people can get it. Git is the tool; GitHub is the place. They are different things.
>
> Git works in four places. Let me draw them.



Instructor Note: VS Code's Source Control panel offers buttons for these steps. Today we use the terminal commands so learners see the four places Git works in. Tell them the buttons do the same thing and can be used later.

Place	What it is
Working folder	The files you see and edit.
Staging	A waiting room: the changes you have chosen to include in the next snapshot.
Local history	Snapshots (commits) saved on your computer, each with a message.
GitHub (remote)	The copy online. Others clone from here, and you push to it.

> **Analogy to use**
> Taking a group photo. You choose who stands in the frame (git add), take the photo with a caption (git commit), and only then post it to the shared album (git push). Teammates download the album with git pull.

Step 1: tell Git who you are (once per computer)

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Use the email on your GitHub account, or the private "noreply" email that GitHub offers in its settings. This is the author name attached to every commit.

Step 2: start a repository

```
git init -b main
git status
```

What Git prints

```
Initialized empty Git repository in .../meridian-workbench/.git/
On branch main
No commits yet
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .env.example
        .gitignore
        config/
        data/
        fixtures/
        pyproject.toml
        requirements.txt
        src/
        tests/
```

“Untracked” means Git sees the file but is not recording it yet. Once you git add and commit, it becomes tracked. Ignored files do not appear at all. That is why .venv is missing from this list.

Your list will differ slightly. Look for two things: .venv and outputs are not listed, and neither is .env. That is .gitignore doing its job. Prove it with a harmless secret:

```
git check-ignore -v .env
```

Git prints the .gitignore line that matched. If it prints nothing, .env is not ignored yet. Stop and fix .gitignore before you add anything.

Step 3: stage, then commit

```
git add .
git status
git commit -m "Create reproducible Meridian Workbench V0"
git log --oneline
```

What each piece does

- git init -b main creates the hidden .git folder, which holds all history, and names the first branch main. Branches are a later topic; we stay on main.
- git status is the question you ask before and after everything. Develop the habit: status, then act, then status.
- git add . stages all changes in this folder. *(Why not save everything in one go? Staging lets you choose what goes into a snapshot. You might have edited five files but want this snapshot to be about one change. It is also your last look at the list before anything is saved.)* After it, git status shows the files under "Changes to be committed". Read that list: it must not contain .venv, .env or outputs.
- git commit -m "message" saves the snapshot with a message. The message is for the next person, who reads the history.
- git log --oneline shows the history as one line per commit.

> **Pause and predict**
> After git add . and before git commit, ask: "Is there anything on this list that should not be going to the internet?" Make them read the list file by file. This habit prevents most accidents.

> **Instructor note: commit messages**
> Good commit messages say what and why in a short line: "Add settings file and batch-date override". Bad ones: "stuff", "fix", "final2". Show one of each and ask which one helps a stranger.

Step 4: publish to GitHub

Why GitHub if we have Git: Git keeps history on your computer. If the laptop breaks, or a colleague needs the code, a local history helps nobody. GitHub is the shared, backed-up copy others can clone.

- In GitHub, click New repository. Name it meridian-workbench. Choose Public.
- Do not tick "Add a README", ".gitignore" or a licence. We have our own, and an empty repository avoids a clash.
- Copy the repository URL it shows, which looks like:

  https://github.com/your-username/meridian-workbench.git

```
git remote add origin https://github.com/<your-username>/meridian-workbench.git
git push -u origin main
```

What each piece does

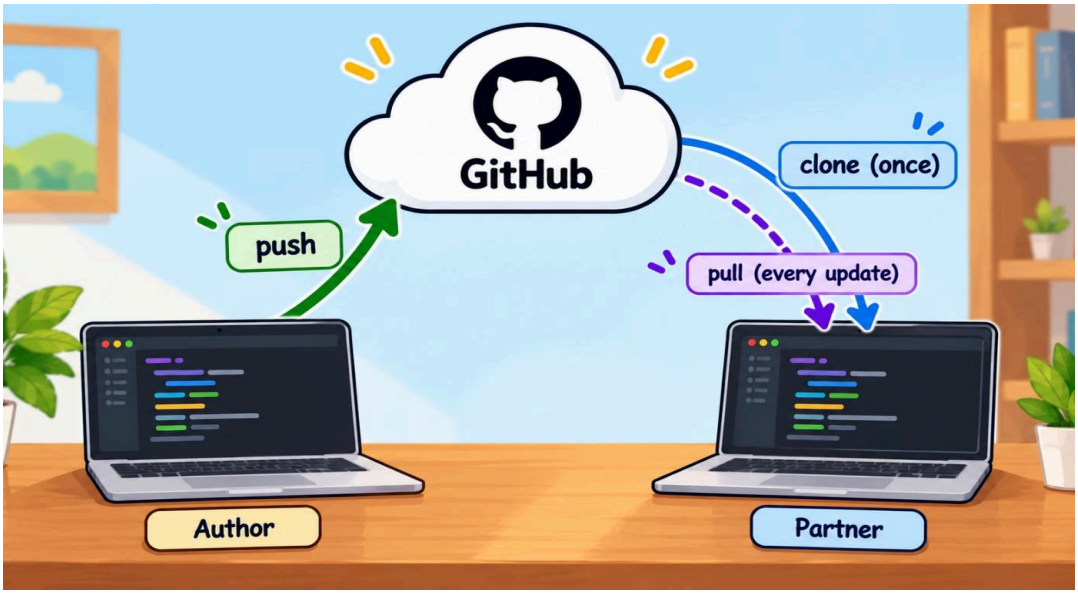
- git remote add origin URL gives the online copy a short name, origin. A remote is just a saved address.
- git push -u origin main sends the main branch to origin. -u links the two, so later you can type just git push.
- Refresh the GitHub page. The files appear, with src, tests, data and the rest. There is no .venv and no .env.

> **Platform note: signing in to GitHub**
>
> GitHub no longer accepts your account password in the terminal. Sign in with Git Credential Manager (installed with Git for Windows and offered on macOS), the GitHub CLI (gh auth login), or a personal access token pasted when asked for a password. Have the room use whichever one you tested in prep.

> **Common gotcha**
>
> - Public means everyone can read it, including the history. Nothing secret goes in, ever. In this class there is nothing secret, which is the safe way to practise.
> - If a key was ever committed, deleting it later does not remove it from history. Replace the key. This is why we check the staging list before every commit.

Step 5: clone and pull (instructor demo, 3 minutes)

> Now watch what a colleague does. They do not start from my laptop. They clone, which means "make a full copy, with history, from GitHub".




```
cd ..
git clone https://github.com/<your-username>/meridian-workbench.git meridian-clone
cd meridian-clone
git log --oneline
```

The history is there. Now go back to the original folder, add a line to a file (for example, a comment at the top of settings.toml), then:

```
git add .
git commit -m "Clarify settings comment"
git push
```

And in the clone:

```
git pull
```

What each piece does

- git clone URL folder copies the whole repository and remembers where it came from. You do it once.
- git pull fetches new commits from GitHub and brings them into your folder. You do it often, especially before you start work. If you and a teammate changed the same lines, pull stops and asks you to choose; that is a conflict, and we handle conflicts when we learn branches.
- Direction is the whole idea: commit and push send your work out. pull brings others' work in.

> Clone once, pull often, commit then push. If you remember only that, you can work with a team.

Check for understanding

Ask: What is the difference between git commit and git push?
Answer: Commit saves a snapshot on your computer. Push sends the saved commits to GitHub.
Ask: Why do we run git status before git commit?
Answer: To check that the staging list contains only what we mean to share, and nothing secret.

Common mistakes to expect

- Running git init inside a folder that is already inside another repository.
- git init -b main fails with "unknown switch 'b'": the Git version is older than 2.28. Run git init, then git branch -M main.
- Creating the GitHub repository with a README, then getting a "rejected" error on the first push because the histories differ. Make it empty.
- Using git add . before .gitignore exists, which stages .venv. Undo with git rm -r --cached .venv, then fix .gitignore.
- Forgetting -m, so Git opens a text editor that nobody knows how to leave. In vim: press Esc, type :q! and Enter, then retry with -m.
- Pushing before committing. Nothing is sent, because nothing was saved.
- Pasting the URL of the web page instead of the .git URL, or a typo in the username.
- Expecting git pull to upload anything. Pull only brings changes in.

If learners struggle

If sign-in blocks someone, pair them with a neighbour for this section and have them watch. Fix their sign-in in the break, or during the lab. The commit part (Steps 1 to 3) still works fully offline, so they have not lost the main lesson.

Quiz 4 of 6 | After Section 9 (minute 93) | Multiple choice (one answer) | about 90 seconds

Checks: Reading git status.

Question. After a commit, git status shows the message below. What does it mean and what is the next step?

```
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)
```

A. GitHub has a commit you do not have, so run git pull

B. Your changes are not saved yet, so run git add

C. You have a commit saved on your computer that GitHub does not have yet, so run git push

D. The repository is damaged and must be cloned again

Correct answer: C

Commit saves a snapshot locally. Push sends it to GitHub. "Ahead" means local has something the remote does not. Note that git status does not check GitHub live: it compares with what Git last downloaded. If a teammate pushed since then, you can be ahead and behind at once, and your push is rejected until you git pull.

A mixes up ahead with behind. B describes uncommitted work, which status would list under changes.

Instructor tip: Draw the four places from Section 9 on the board and point at the gap between "Local history" and "GitHub".

Transition

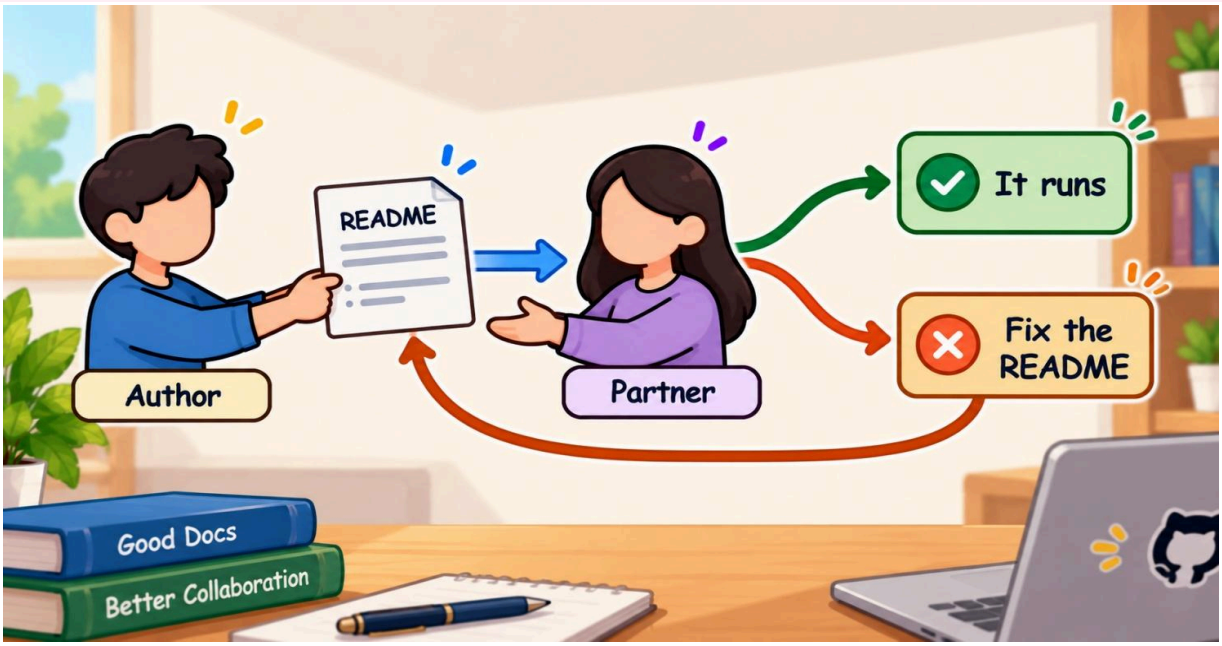
> The code is online. But a stranger who finds it still would not know what to do. That is the job of the README.

Section 10: README and the stranger test

Duration	12 minutes (minute 93 to 105), including Quiz 5
Teaching objective	Write a README that tells another engineer how to install, run and check the project, and test it on a partner who has not seen the code.

Say

> A README is the front page of your repository. It answers one question for someone who knows nothing: how do I run this? It is the written version of everything we have done for the last 90 minutes. If it is wrong, a colleague loses an hour. If it is right, they lose five minutes.
>
> There is no point trusting a README that only its author has read. So we will test it on a stranger: your partner.



Step 1: write README.md (3 minutes)

Create README.md in the project root. Learners write their own version using the outline below.

The `.md` ending means **Markdown**, a simple way to write headings and lists using plain characters, which GitHub turns into a nicely formatted page. Commands go in code blocks so a stranger can copy and paste them with no typos.

Section	What it must say
What this is	One or two sentences. Which cases need attention, with evidence.
Requirements	Python 3.11 or newer, Git, and internet access for the first install. Nothing else.
Set up	Clone, create the environment, activate it (both systems), install from requirements.txt.
Run	The command, and the expected output (the four flagged cases and the count).
Check	Run the rule checks and ruff. Say what success looks like.
Settings	Where settings live, and how to override the batch date with an environment variable.
Project layout	A short folder tree with a one-line purpose for each folder.

> Write commands exactly as you would type them, in code blocks, with the expected output. "Install the dependencies" is not an instruction. python -m pip install -r requirements.txt is.

```
git add README.md
git commit -m "Add README with setup and run steps"
git push
```

Step 2: the stranger test (6 minutes)

- Partners swap GitHub URLs.
- The partner opens a new terminal, goes to an empty folder, and clones the author's repository.
- The partner follows only the README. No asking the author, no looking at the code, no help from memory.
- The author watches in silence and writes down every place the partner hesitates or has to guess.
- Success means the partner runs python -m meridian_workbench and sees 4 of 7 cases need attention.

Step 3: fix and pull (2 minutes)

- The author fixes the README at the first gap, commits and pushes.
- The partner runs git pull and sees the README change arrive, which is the pull practice from Section 9.

Pause and predict

Before the test starts ask: "How many of you think your README is complete?" Most hands go up. After the test ask again. The gap is the lesson.

Instructor note

Typical gaps: no activation command, only one operating system covered, Python version not stated, missing the requirements.txt step, or an expected-output block that does not match. Circulate and note which ones recur, then name them to the whole room.

Check for understanding

Ask: Why test the README on someone who has not seen the code?

Answer: Because the author already knows the missing steps and cannot see them. A stranger finds the gaps.

Ask: A partner gets ModuleNotFoundError. What do you check first?

Answer: Whether the environment is activated, and whether pip install -r requirements.txt ran.

Common mistakes to expect

- Writing "install the requirements" without the actual command.
- Covering only macOS/Linux or only Windows.
- Telling the partner to copy a .env file that is not in the repository. Use .env.example, and note it is optional.
- Leaving out the expected output, so nobody knows whether it worked.
- The author helping the partner. Silence is part of the test.

If learners struggle

If a learner has no partner, they run the test on you or on a clean folder of their own: clone into a new directory, create a fresh .venv there, and follow the README literally.

Quiz 5 of 6 | After Section 10 (minute 105) | Multiple Choice Questions | about 90 seconds

Checks: Debugging a fresh clone.

Question. A partner clones your repository, sets it up and runs python -m meridian_workbench. They see ModuleNotFoundError: No module named 'meridian_workbench'. Which ONE is the most likely cause?

A. Their `.venv` was not activated, so a different Python ran the command

B. `data/cases.csv` is missing from the repository

C. They did not create a `.env` file from `.env.example`

D. Their Python version is too new

Correct answers: A

A is the most likely cause because the project is only importable inside the Python that received the editable install. If `.venv` is not active, a different Python runs the command and cannot find the package, which gives exactly this error.

B would give a `FileNotFoundError`. C would only matter after the import works, and it would show up as a missing-key error, not `ModuleNotFoundError`. D is unlikely for a "module not found" message.

Instructor tip: This is the error you will see most in real life. Tell learners to read the last line of an error first.

Transition

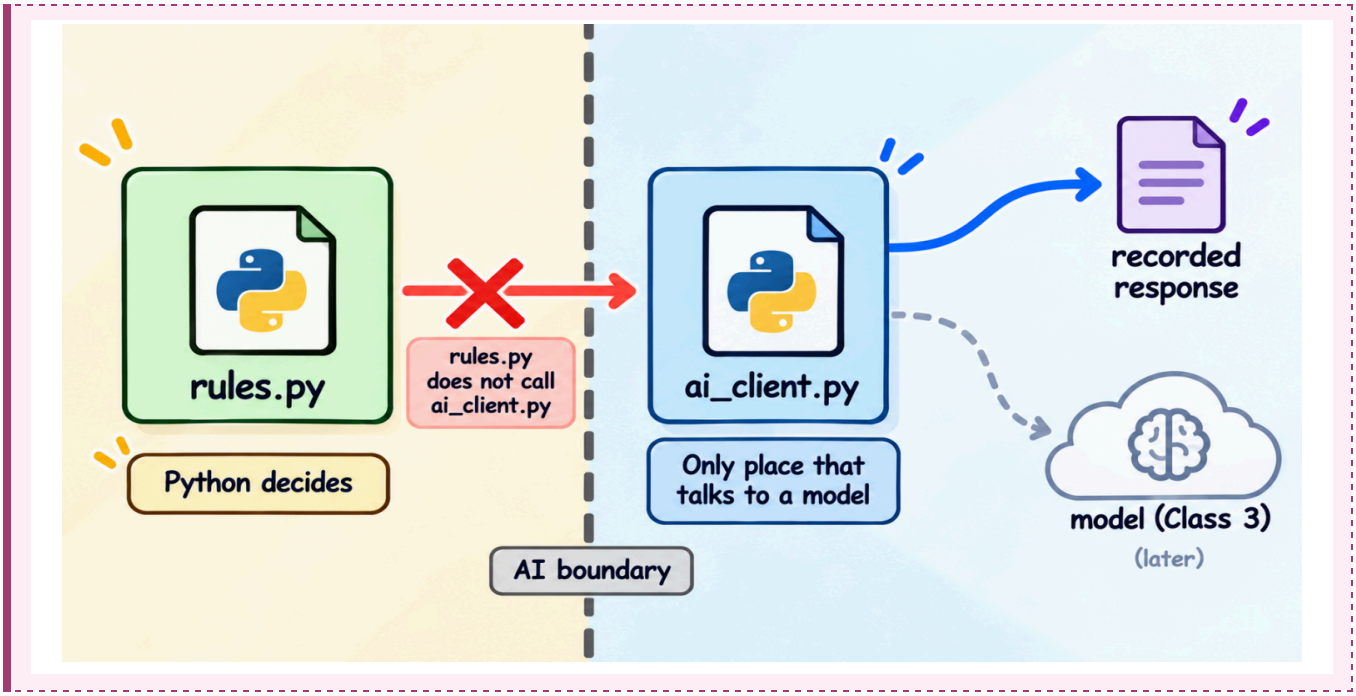
> One last piece before we close. Soon a model will suggest labels for the notes. Where should that code live, and what must never touch it?

Section 11: Prepare the AI boundary

Duration	8 minutes (minute 105 to 113), including Quiz 6
Teaching objective	Create the single, clearly named place where model access will live, with a recorded-response path so setup problems can never block the class.

Say

> Class 3 adds a model that reads the free-text notes and suggests an issue label with evidence. We are not calling one today. But the biggest mistake teams make is to scatter model code and keys through the program. We will draw the line first, while the project is still small.



Design decision	Why
One file, ai_client.py, owns all model access	Anyone can find the boundary in one place, and replace the provider without touching the rules.
A setting, ai mode, defaults to "recorded"	No key, no network and no provider account are needed to run the project or the class.
A fixture file holds saved responses	Learners see realistic suggestions today, and tests stay stable later.
rules.py never imports ai_client	Python decides who needs attention. The model only suggests what a note means.
The API key has a placeholder only	.env.example has a commented-out line. Nothing real, and nothing used until Class 3.

Step 1: add the setting

Append to config/settings.toml:

```
[ai]
# "recorded" reads saved responses from fixtures/. Live calls arrive in Class 3.
mode = "recorded"
```

In config.py, read it next to the batch date and return it. Add one line before return, and one entry in the dictionary:

```
    ai_mode = os.environ.get("MERIDIAN_AI_MODE", raw["ai"]["mode"])

    return {
        ...
        "ai_mode": ai_mode,
    }
```

Step 2: create src/meridian_workbench/ai_client.py

```
"""The one place where the Workbench talks to a model.

Class 2 supports recorded responses only. The rules never import this file.
"""

import json


def get_suggestion(note_id, settings):
    """Return a suggestion for one note. The model only suggests, it never decides."""
    mode = settings["ai_mode"]
    if mode == "recorded":
        return _read_recorded(note_id, settings["fixtures_dir"])
    raise NotImplementedError(f"AI mode '{mode}' arrives in Class 3. Use 'recorded' for now.")


def _read_recorded(note_id, fixtures_dir):
    with open(fixtures_dir / "recorded_responses.json", encoding="utf-8") as f:
        responses = json.load(f)
    if note_id not in responses:
        raise KeyError(f"No recorded response for {note_id} in recorded_responses.json")
    return responses[note_id]
```

What each piece does

- get_suggestion is the only function the rest of the program will call. It takes a note ID and the settings, and returns a suggestion.
- In recorded mode it reads fixtures/recorded_responses.json and returns the saved answer for that note. No internet, no key.
- Any other mode raises NotImplementedError with a clear message. That is honest: the feature does not exist yet, and the error tells you where it will arrive.
- A note with no recording raises a KeyError that names the note, instead of quietly returning nothing. Same idea: a clear error is better than a silent failure.
- _read_recorded starts with an underscore: a Python habit meaning "internal to this file, do not call it from outside".

Open fixtures/recorded_responses.json and read all three aloud. N-01 has an issue family, a summary and an evidence span copied word for word from the note, and review_required is false. N-02 abstains, with a reason, because "see previous" says nothing. N-03 reads a cryptic note, "cust n/a x2, rtn to dpt", and its summary is an interpretation ("suggests the customer was unavailable twice"), so review_required is true: a human should check it. N-01 and N-03 have no abstained key at all, which simply means they did not abstain. A good suggestion points to evidence, says when a human should check, and is allowed to say "I do not know".

Step 3: prove the boundary with a check

```
"""Checks for the AI boundary: recorded mode works and the rules stay free of it."""

import subprocess
import sys

from meridian_workbench.ai_client import get_suggestion
from meridian_workbench.config import load_settings
from meridian_workbench.loaders import load_notes

RULES_ONLY = "import sys, meridian_workbench.rules; print('meridian_workbench.ai_client' in
sys.modules)"

def rules_load_ai_client():
```

```
    """Start a fresh Python, import only the rules, and report whether ai_client came along."""
    result = subprocess.run(
        [sys.executable, "-c", RULES_ONLY], capture_output=True, text=True, check=True
    )
    return result.stdout.strip() == "True"

def run_checks():
    settings = load_settings()
    notes = {n["note_id"]: n for n in load_notes(settings["data_dir"] / "notes.json")}

    suggestion = get_suggestion("N-01", settings)
    assert suggestion["issue_family"] == "ADDRESS_CONFIRMATION"
    assert suggestion["evidence_span"] in notes["N-01"]["text"]

    abstained = get_suggestion("N-02", settings)
    assert abstained["issue_family"] is None
    assert abstained["review_required"] is True

    assert not rules_load_ai_client()
    print("AI boundary checks passed")

if __name__ == "__main__":
    run_checks()
```

```
python tests/check_ai_boundary.py
```

What Python prints

```
AI boundary checks passed
```

- The first block proves the evidence span is quoted from the note, not invented. (It does not prove the label is right; judging that is Class 3's evaluation work.)
- The second proves abstention is preserved and flagged for review.
- The last check starts a fresh Python (the same python -c trick from Section 3), imports only rules.py, and asks whether ai_client came along. A fresh Python matters: this test file already imported ai_client, so checking here would always say yes. If somebody wires the model into the rules next month, directly or through another module, this check fails.

Step 4: see the future error, then commit

macOS or Linux

```
MERIDIAN_AI_MODE=live python tests/check_ai_boundary.py
```

Windows PowerShell

```
$env:MERIDIAN_AI_MODE = "live"
python tests/check_ai_boundary.py
Remove-Item Env:MERIDIAN_AI_MODE
```

You will see NotImplementedError: AI mode 'live' arrives in Class 3. Use 'recorded' for now. A clear error is better than a silent failure. Now ruff and commit:

```
ruff format .
ruff check .
git add .
git status
git commit -m "Add AI boundary with recorded responses"
git push
```

Add one line to the README under Settings: the AI mode is recorded by default and no key is needed. Commit that too.

Analogy to use

A hospital pharmacy window. Everything that goes in or out passes the one window. Doctors do not walk into the storeroom. If the supplier changes, only the window changes.

Check for understanding

Ask: Why does the default AI mode stay recorded?

Answer: So setup problems, a missing key or a provider outage can never stop someone from running the project.

Ask: Which file must never mention ai_client?

Answer: rules.py. Python decides, the model only suggests.

Common mistakes to expect

- Putting a real API key in .env.example. It is committed and public.
- Importing ai_client inside rules.py "just to try". The boundary check will fail, even if the import is hidden inside another module that rules.py imports. That is its job.
- Misspelling the fixture filename or placing it outside fixtures/.
- Treating recorded responses as real model output. They are a stand-in; say so out loud.
- Adding provider code, a model name or an SDK import. That is Class 3.

Quiz 6 of 6 | After Section 11 (minute 113) | Multiple choice (one answer) | about 90 seconds

Checks: Where model access belongs.

Question. Which design best keeps the AI boundary clean for the Workbench?

A. Put the API key in rules.py so that needs_attention can ask the model

B. Call the model inside needs_attention and fall back to a default on failure

C. Make a live model call mandatory so that setup fails early if it is missing

D. Keep all model access in ai_client.py, default to recorded mode, and never import ai_client from rules.py

Correct answer: D

The rules decide, so they must not depend on a model, a key or a network. One clearly named client file keeps the boundary visible, and recorded mode means a missing key or a provider outage cannot block the class.

A and B mix a secret and a model into domain rules. C makes setup depend on something outside the learner's control.

Instructor tip: Connect back to the Class 1 line: Python decides, the model only suggests.

Transition

> That completes the build. Let us put it into words, because explaining it is how you know you own it.

Section 12: Say it, then close

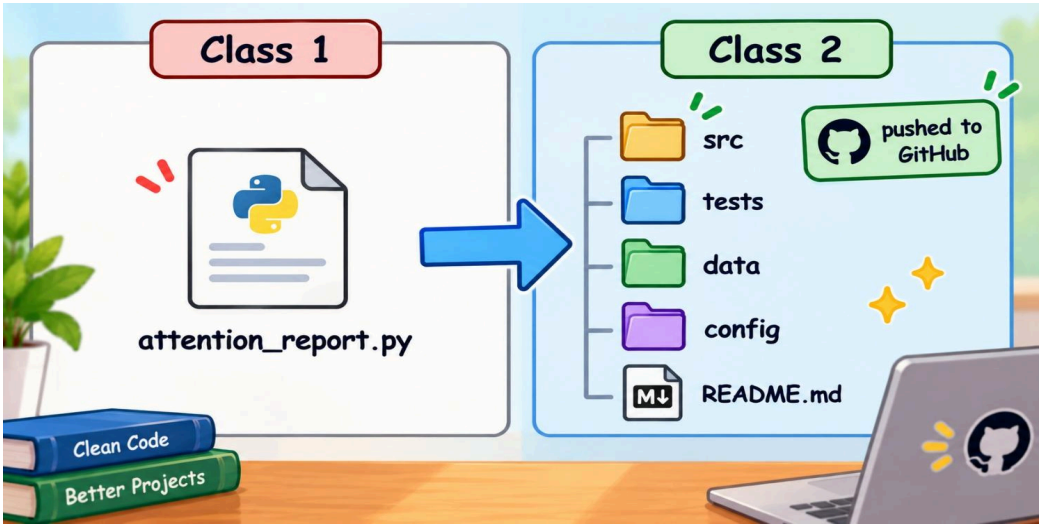
Duration	7 minutes (minute 113 to 120)
Teaching objective	Learners explain the repository in three sentences, answer the opening question again, and see what comes next.

Do: three sentences (2 minutes)

> Turn to your partner. In three sentences, explain what you built today to someone who was not here. Use these starters if you like: "The project is organised as...", "Someone else can run it by...", "The model will be kept in..." Then swap.

Say: revisit the opening question

> At the start I sent my file to a stranger and it failed. Now it is a repository with a README that your partner followed to a working run. Look at what changed.



Question	Before (Class 1)	After (today)
Where does the code live?	One file	src/meridian_workbench, with one job per module
Which Python and packages?	Whatever is on the laptop	.venv, versions pinned in pyproject.toml, one install line in requirements.txt
How do you change the date?	Edit the code	Settings file, or an environment variable
Where do secrets go?	Nowhere planned	.env, ignored by Git, with a .env.example template
How do others get it?	Send the file	git clone from GitHub
How do they know how to run it?	They ask you	The README
How do we know it is right?	Checks inside the script	tests/ plus ruff

Do: exit ticket (2 minutes)

Ask learners to answer in chat, one line each:

- One command you now understand that you did not at the start.
- One step in the README that your partner struggled with.
- One question you still have.

Say: Class 3 preview (1 minute)

> Next class we give the model its job: read a messy note and suggest an issue label with evidence, or say it cannot tell. We will meet pytest to test properly, type checking to catch mistakes early, and we make our first live call. All of it goes through the one door we built today: ai_client.py.

Check for understanding

Ask: What do you need to run the project on a new laptop?

Answer: Python 3.11 or newer, Git, then clone, create and activate .venv, install from requirements.txt, and run python -m meridian_workbench.

Common mistakes to expect

- Learners who did not finish a step leaving without the code on GitHub. Check that the final push happened; the repository is the deliverable.
- Leaving a real key anywhere in the repository. Do a final git log and a look at the files before everyone leaves.
